



MEMOIRE DE FIN D'ETUDES

présenté à

La Faculté des Sciences de Sfax

en vue de l'obtention du

DIPLOME DE LICENCE EN

SCIENCE DE L'INFORMATIQUE

par

Racem BOUAICHA

DEVELOPPEMENT D'UNE APPLICATION DE GESTION DE CAS DE TEST ET DE GENERATION DES RAPPORTS DE TESTS

soutenu le 5 Juin 2023, devant le jury composé de :

Mme. Sirine REBAI

Président

Mme. Amal BEN HMIDA

Examineur

Mr. Mohamed MHIRI

Encadrant Académique

Mlle. Sirine MARRAKCHI

Co-encadrante Académique

Mr. Mohamed GASMI

Encadrant Industriel

Stage réalisé à « SiFast »



Dédicaces

Je dédie ce mémoire, fruit d'un mon long chemin d'étude,

A mon père Hafedh

Pour toutes les privations et sacrifices consentis, pour le soutien moral et financier durant mes années d'études, je le remercie pour l'homme qu'il est et pour ce qu'il m'a fait devenir.

A ma mère Rawdha

Aucune dédicace n'exprime mes sentiments, que Dieu la préserve et lui procure santé et longue vie. Elle m'a donné la vie, la tendresse et le courage pour réussir. Tout ce que je peux lui offrir ne pourra exprimer l'amour et la reconnaissance que je lui porte.

A mes frères Mohamed et Firas et mes sœurs Soulaïma et Farah

Pour leurs conseils, leurs soutiens moral et matériel sans faille.
Que Dieu les Protège, et leur procure la réussite dans leurs vies privées.

A mes chers amis,

Merci d'être toujours à mes côtés, par votre présence et votre réactivité.
Merci pour les beaux et inoubliables moments que nous avons partagés ensemble et pour le bonheur que j'ai apprécié avec vous.
A tous ceux qui m'ont enseigné.

Remerciements

Je tiens à remercier DIEU, pour m'avoir donné la patience et le courage pendant ces années d'étude.

Au terme de ce travail, je tiens à adresser mes profonds et sincères remerciements à mes encadrants universitaire **Monsieur Mohamed MHIRI** et **Mademoiselle Sirine MARRAKCHI**, pour leur disponibilité, leurs conseils généreux et de m'avoir fait confiance tout au long de ce travail de Projet de Fin d'Etudes.

Je suis particulièrement reconnaissant envers **Monsieur Anis SAMET** et **Monsieur Wissem HAMZA** pour leurs accueils à **SIFAST** et aussi mon encadrant professionnel **Monsieur Mohamed GASMI** qui a présenté une aide vitale, inestimable et une collaboration complète tout au long de l'élaboration de ce mémoire.

Mes sincères remerciements vont également, aux **Madame Sirine REBAI** et **Madame Amal BEN HMIDA** pour l'intérêt qu'ils ont porté à ma recherche en acceptant d'examiner mon travail et de l'enrichir par leurs propositions.

Je tiens aussi, à remercier ma chère famille et mon binôme dans la société « Youssef MRABET » pour leur contribution, leur soutien et leur patience.

Enfin, j'adresse mes sincères remerciements à tous mes proches et amis, qui m'ont toujours soutenu et encouragée au cours de la réalisation de ce mémoire.

A vous tous, MERCI BEAUCOUP

TABLE DES MATIERES

<i>Dédicaces</i>	<i>ii</i>
<i>Remerciements</i>	<i>iii</i>
<i>Table des matières.....</i>	<i>iv</i>
<i>Liste des figures.....</i>	<i>vii</i>
<i>Liste des tables</i>	<i>ix</i>
<i>Liste des acronymes</i>	<i>x</i>
<i>Introduction générale.....</i>	<i>1</i>
Chapitre 1 Étude préalable.....	3
1 Introduction.....	3
2 Présentation de l'organisme d'accueil	3
2.1 SiFAST	3
2.2 Missions.....	4
2.3 Modèle de livraison Front/Back	4
3 Présentation du projet	5
3.1 Assurance qualité (QA)	5
3.2 Test logiciel	5
4 Étude de l'existant.....	6
4.1 Analyse de l'existant	6
5 Solution proposée.....	9
6 Méthodologie de développement : méthodologie Agile	10
6.1 Présentation des méthodes agiles	10
6.2 Méthode Scrum.....	11
7 Tableau Backlog du product	12
7.1 Identification des acteurs	12
7.2 Fonctionnalités du tableau backlog product	12
7.3 Planification du projet	16
8 Conclusion	16

Chapitre 2	Sprint 1 - Gestion des utilisateurs	17
1	Introduction	17
2	Analyse	17
3	Conception	18
3.1	Diagramme de séquence du cas gérer compte	18
3.2	Diagramme de séquence du cas d'utilisation « inscrire »	19
3.3	Diagramme de classes du premier sprint	20
4	Modèle relationnel	21
5	Réalisation	21
5.1	Interface de création d'un compte et interface de connexion	22
5.2	Interface pour afficher la liste des utilisateurs	22
6	Conclusion	22
Chapitre 3	Sprint 2 - Préparation du plan de test	23
1	Introduction	23
2	Analyse	23
3	Conception	24
3.1	Diagrammes de séquence du cas d'utilisation « gérer les projets »	24
3.2	Diagramme de séquence du cas d'utilisation « gérer les sprints »	25
3.3	Diagramme de séquence du deuxième sprint « gérer les cas d'utilisation »	26
3.4	Diagramme de séquence du deuxième sprint « gérer les cas de test »	29
3.5	Diagramme de classes	30
4	Modèle relationnel	31
5	Réalisation	31
5.1	Interfaces pour la gestion des projets de test	31
5.2	Interfaces pour la gestion des sprints de test	32
5.3	Interfaces pour la gestion des cas d'utilisation	33
5.4	Interfaces de gestion d'un cas de test	34
6	Conclusion	34

Chapitre 4 Sprint 3 - Exécution du plan de test.....	35
1 Introduction.....	35
2 Analyse	35
3 Conception	36
3.1 Diagramme de séquence du cas d'utilisation « proposer un scénario »	36
3.1 Diagramme de séquence du cas d'utilisation « exécuter un scénario ».....	36
3.2 Diagramme de classe	38
4 Modèle relationnel	40
5 Réalisation.....	40
5.1 Interfaces de liste des scénarios	40
5.2 Interfaces de liste des cas de test exécuté ou à exécuter.....	40
5.3 Interfaces de l'exécution de cas de test	41
5.4 Exemple de rapport.....	41
6 Conclusion	42
 Chapitre 5 Clôture du projet.....	 43
1 Introduction.....	43
2 Architecture du système développé	43
2.1 Architecture globale	43
2.2 Fonctionnement d'application	44
3 Outils et environnements de travail	45
3.1 Environnement logiciel.....	45
3.2 Environnement de développement	45
4 Evaluation	46
5 Conclusion	46
Conculsion générale	47
 <i>Références bibliographiques</i>	 <i>48</i>

LISTE DES FIGURES

Figure 1: Modèle de livraison Front/Back	4
Figure 2: Aperçu sur TestRail	7
Figure 3: Aperçu sur Qase.....	8
Figure 4: Aperçu sur Test Collab	8
Figure 5: Processus de SCRUM.....	11
Figure 6: Diagramme de cas d'utilisation de premier sprint.....	17
Figure 7: Diagramme de séquence « gestion de comptes utilisateurs ».....	19
Figure 8: Diagramme de séquence « inscrire »	20
Figure 9: Diagramme de classe de premier sprint.....	21
Figure 10: Interface d'inscription.....	22
Figure 11: Interface de Login.....	22
Figure 12: Interface affichant la liste des utilisateurs	22
Figure 13: Diagramme de cas d'utilisation de sprint 2	23
Figure 14: Diagramme de séquence du deuxième sprint « Gérer les projets ».....	25
Figure 15: Diagramme de séquence du deuxième sprint « gérer les sprints »	27
Figure 16: Diagramme de séquence du cas d'utilisation « gérer les cas d'utilisation »	28
Figure 17: Diagramme de séquence du deuxième sprint « gérer cas de test ».....	29
Figure 18: Diagramme de classe du deuxième sprint	31
Figure 19: interface liste projet de test	32
Figure 20: Interface de création projet de test.....	32
Figure 21: Interface liste des sprints de test	32
Figure 22: Interface de création de sprint.....	33
Figure 23: Interface affichage de la liste des cas d'utilisation.....	33
Figure 24: Interface de création de cas d'utilisation.....	33
Figure 25: Interface liste des cas de tests	34
Figure 26: Interface de création de cas de test	34
Figure 27: Diagramme de cas d'utilisation du troisième sprint	35
Figure 28: Diagramme de séquence du troisième sprint (Proposer scénario).....	37
Figure 29: Diagramme de séquence du troisième sprint (Exécuter un scénario).....	37
Figure 30: Diagramme de classe du troisième sprint	39

Figure 31: Interface de liste des scénarios	40
Figure 32: Interface de la liste cas de test exécuté ou à exécuter.....	41
Figure 33: Interface d'exécution de cas de test.....	41
Figure 34: exemple de rapport	42
Figure 35: Architecture global	43
Figure 36: Démarche générale	44

LISTE DES TABLES

Table 1 : Comparaison entre les logiciels	9
Table 2 : Identification des acteurs	12
Table 3: 'Backlog du projet'.....	13
Table 4 : Planification de durées	16
Table 5 : Description de séquence (Gérer compte)	18
Table 6: Scénario de la séquence (inscrire).....	20
Table 7 : Description des classes (sprint1).....	21
Table 8 : Description de la séquence (Gérer projet de test)	24
Table 9 : Description de la séquence (gérer sprint).....	26
Table 10: Description de la séquence (Gérer cas d'utilisation)	26
Table 11 : Description de la séquence (Gérer cas de test)	29
Table 12 : Description de classes (sprint 2)	30
Table 13 : Description de la séquence (Gérer scénario).....	36
Table 14 : Description de la séquence (Gérer scénario).....	38
Table 15 : Description de classe sprint 3.....	38

LISTE DES ACRONYMES

BI	Business Intelligence
CMMi	Capability Maturity Model Integration
CMS	Content Management System
HTTP	Hypertext Transfer Protocol
ISO	International Organization for Standardization
JEE	Java Enterprise Edition
LAMP	Linux, Apache, MySql/MariaDB and Php/Perl/Python
PHP	Hypertext Preprocessor
QA	Quality Assurance
REST	Representational State Transfer
UML	Unified Modeling Language

INTRODUCTION GENERALE

Le développement des logiciels est un travail collaboratif entre les membres d'une équipe au sein d'une organisation. Il nécessite l'utilisation d'une ou plusieurs méthodes de gestion de projet. L'efficacité d'une méthode bien déterminée dépend de la souplesse de l'exécution de ses étapes et une maîtrise des outils et techniques correspondants. Actuellement, les méthodes agiles sont très utilisées dans le monde de développement. Elles possèdent plusieurs avantages et donnent des résultats satisfaisants. Elles favorisent les relations humaines et des techniques de communication avancées. En plus, elles permettent de découper un projet en modules courts faciles à développer.

Cependant, les développeurs ont besoin d'un moyen pour s'assurer de la qualité des livrables et leur taux de satisfaction par rapport aux exigences des clients. Rappelons que l'assurance qualité, selon la définition ISO 9000, est la partie du management qui donne confiance en ce que les exigences de la qualité soient satisfaites. Elle constitue une garantie de la conformité du logiciel aux normes de la qualité et s'identifie sous la forme d'un document écrit, qui présente toute la politique qualité suivie tout au long de la production. Elle se base sur l'ensemble de tests effectués aux logiciels.

Dans ce contexte et dans le cadre de stage de fin d'année, la société Sifast nous propose de développer une application pour automatiser de gestion de tests. Cette application a pour rôle de simplifier les tâches du service QA en tenant compte en premier lieu de la sécurisation des droits d'accès dans les projets de tests, de générer des rapports de tests et de suivre des tests. C'est une application générique qui se base sur les concepts de la méthode SCRUM. Nous citons comme exemple, le projet, le sprint, les cas d'utilisation. Elle représente une assistance des tâches effectuées par les testeurs.

Ce rapport est organisé en cinq chapitres.

- ✓ Le premier chapitre présente une étude préalable du projet. Il présente la société encadrante, donne quelques exemples de logiciels existants.
- ✓ En plus, expose la solution proposée et il donne ses différentes fonctionnalités.
- ✓ Ces fonctionnalités sont réparties en sprints. Ces sprints seront développés dans le deuxième, le troisième et le quatrième chapitre.
- ✓ Dans le cinquième chapitre, nous présentons l'architecture et les environnements utilisés. Enfin, nous terminons par une conclusion et la proposition de quelques perspectives.

CHAPITRE 1 ÉTUDE PREALABLE

1 Introduction

Dans ce chapitre, nous présentons notre projet de fin étude effectué dans de la société SiFAST, Nous commençons par présenter notre société d'accueil. Ensuite, nous donnons un aperçu sur notre projet. Par la suite, nous montrerons le cadre général du projet. Puis, nous présentons quelques applications existantes et leurs limites. Après, nous dévoilerons notre solution et ses fonctionnalités. Enfin, nous exposons la méthodologie Agile utilisée dans le projet en déterminant les acteurs du notre système ainsi que son découpage en sprints.

2 Présentation de l'organisme d'accueil

Le stage de fin d'études a été réalisé au sein de l'entreprise SiFAST pour une période de trois mois. Dans la section suivante, nous donnons une description de la société.

2.1 SiFAST

SiFAST est une société de services numériques créée en 2010 par des professionnels du service et de l'informatique, c'est un des leaders du Nearshore Francophone, qui délivre des services IT qui permettent à ses clients d'achever leur efficacité et leur rentabilité. Grâce à sa maîtrise des nouvelles technologies, son expérience et ses méthodes collaboratives, elle accompagne ses clients dans le développement des solutions sur mesure pour répondre à leurs enjeux axes sur les développements informatiques et l'externalisation sur la base des principes de compétence, de respect et d'innovation, et sur la satisfaction du client. Inspirés des différents standards du marché comme CMMi et ISO 9001 et de l'expérience du Nearshore, elle définit son système qualité tout en assurant son agilité.

2.2 Missions

SiFAST offre à ses clients, en véritable partenaire, un service de qualité et de solutions innovantes, tout en respectant leurs contraintes et leurs spécificités. Elle leur permet d'optimiser leurs ressources en leur garantissant un rapport qualité-prix très compétitif et une flexibilité maximale.

2.3 Modèle de livraison Front/Back

Afin de réaliser sa mission parfaitement, SiFAST a bâti sa politique de ressources humaines autour de l'excellence, convaincue que la force de la société réside dans la qualité et l'expertise des collaborateurs. Elle est à l'écoute de ses salariés, soucieuse de leurs problèmes. Les collaborateurs de SiFAST partagent leurs connaissances, leurs compétences et leurs expériences. Les équipes de SiFAST sont organisées par technologie : LAMP/PHP/CMS, Mobile, .NET, Java JEE, BI. Les responsables d'équipes ont acquis la maîtrise de la gestion de projets near shores complexes et peuvent se prévaloir d'une réelle expertise dans les modèles de livraisons Front/Back (voir figure 1).

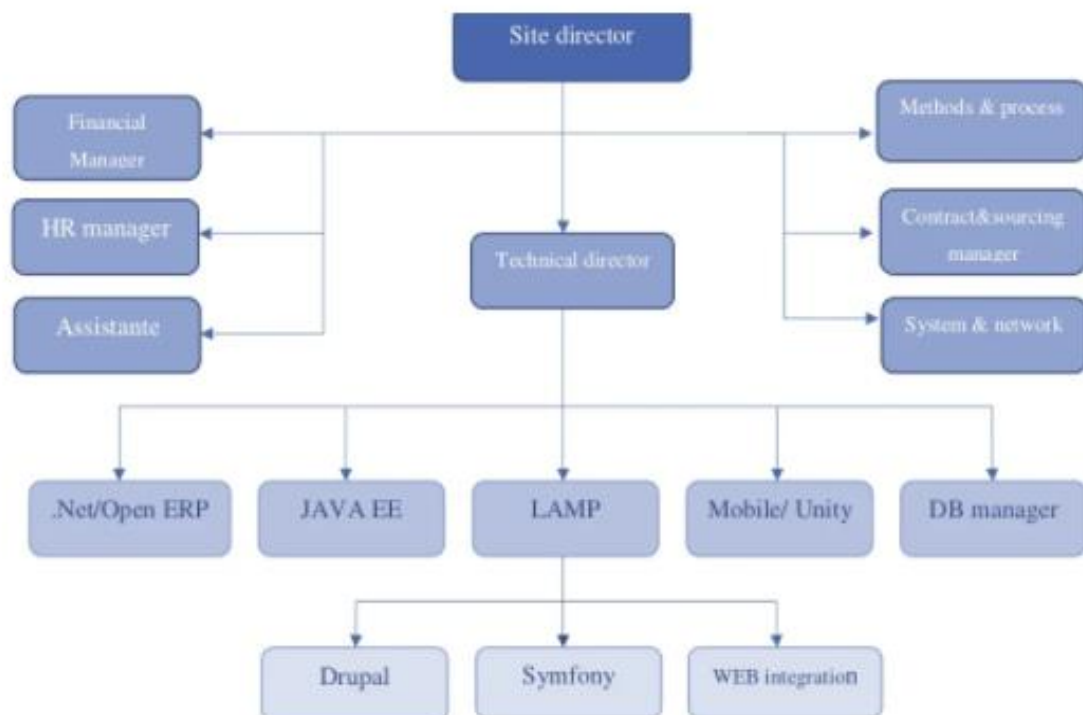


Figure 1: Modèle de livraison Front/Back

3 Présentation du projet

Partant de ses besoins, la société Sifast a décidé de concevoir et de réaliser une application d'automatisation et de génération des rapports de test destiné à l'équipe assurance qualité (QA) et particulièrement la gestion des tests. Elle permet la création et l'archivage des rapports de test pour faciliter les tâches du service QA. Dans les sections suivantes, nous présentons quelques définitions des concepts de base utiles pour notre application.

3.1 Assurance qualité (QA)

L'assurance qualité c'est l'ensemble des actions entreprises pour garantir aux acteurs externes (clients, distributeurs, partenaires...) un niveau de qualité minimum. Dans le cadre de la normalisation, ce niveau est généralement attesté par une norme ISO de la famille ISO9000. L'assurance qualité se compose essentiellement de la rédaction d'un cahier des charges des procédures et d'une démarche de certification du respect de la mise en œuvre de ces procédures et des éléments de la norme ISO choisie. L'opération de l'assurance qualité effectuée dans le cadre des tests de logiciels vise comme objectif de vérifier et garantir que le système logiciel répond à toutes les exigences définies par le client [1, 2].

3.2 Test logiciel

Les tests de logiciels sont avant tout un vaste processus composé de plusieurs processus liés entre eux. L'objectif principal des tests de logiciels est de mesurer la santé des logiciels ainsi que leur exhaustivité en termes d'exigences essentielles. Les tests de logiciels consistent à examiner et à vérifier les logiciels à travers différents processus de test [3]. Le test logiciel nécessite l'utilisation des vocabulaires suivants :

- ✓ **Sprint de test** : le sprint de test est généralement orchestré par un testeur ou une équipe de test spécialisée. Ils travaillent en étroite collaboration avec les développeurs pour comprendre les exigences et les fonctionnalités qui doivent être testées.
- ✓ **Cas d'utilisation de test** : une fonctionnalité est soumise à des tests pour évaluer sa performance, sa fonctionnalité ou sa conformité à des exigences spécifiques. Les cas d'utilisation de test sont utilisés pour déterminer si un logiciel ou un système répond aux attentes et fonctionne comme prévu.

- ✓ **Scénario de test** : c'est le concept qui précise la fonction à tester. La description y est précise et se réfère pour la recette fonctionnelle aux exigences du client. Un scénario de test est composé d'un ou plusieurs cas de tests.
- ✓ **Cas de test** : c'est un chemin fonctionnel à mettre en œuvre pour atteindre un objectif de test. Un cas de test se définit par le jeu d'essai à mettre en œuvre, les étapes des tests sont exécutées et les résultats attendus.

4 Étude de l'existant

A l'heure actuelle, les entreprises informatiques commencent à prendre conscience dans le Management du projet de l'importance de cette phase de test. Cette phase de test est caractérisée par une stratégie de recette fonctionnelle et la mise en œuvre pour assurer que le projet reste dans la bonne direction. Cependant ces entreprises de développement sont obligées de choisir des choix des méthodes et des outils de tests, et faire appel à des professionnels pour l'implémentation de cette stratégie de test.

4.1 Analyse de l'existant

Dans la littérature, il existe des logiciels permettant de gérer les exigences fonctionnelles afin d'assurer une traçabilité entre exigences et plans de test. Par la suite, nous présentons quelques exemples.

- ✓ **TestRail¹** : un logiciel de gestion de cas de test basé sur le web complet pour gérer efficacement, suivre et organiser vos efforts de tests de logiciels. Elle permet de : gérer efficacement les cas, les plans et les essais, augmenter considérablement la productivité des tests et obtenir des informations en temps réel sur la progression de vos tests. TestRail vous aide à gérer et à suivre vos efforts de test logiciel et à organiser votre service d'assurance qualité. Son interface utilisateur intuitive basée sur le Web facilite la création de Scénarios de test, la gestion des tests et la coordination de l'ensemble du processus de test (voir Figure 2).
- ✓ **Qase²** : une plateforme de gestion de tests logiciels qui permet aux équipes de développement de logiciels de créer, de planifier, d'exécuter et de suivre les résultats des tests. Qase offre une gamme d'outils de gestion de tests pour aider les équipes à

¹ <https://test.testrail.net/>

² <https://qase.io/>

gérer efficacement le processus de test, à collaborer et à communiquer, à identifier les problèmes et à résoudre les bugs. Cette plateforme comprend des fonctionnalités telles que la création de cas de test, la gestion des suites de test, l'exécution des tests manuels et automatisés, la génération de rapports, la traçabilité des problèmes et des cas de test, la gestion des utilisateurs et des rôles, et bien plus encore. Les utilisateurs peuvent également intégrer Qase avec d'autres outils de développement, tels que des outils de gestion de projets, des outils de collaboration et des outils de gestion de code source (voir Figure 3).

- ✓ **TestCollab³** : une plateforme de gestion de test logiciel collaborative basée sur le cloud, conçue pour aider les équipes de développement de logiciels à collaborer sur l'exécution et la gestion des tests. Test Collab permet de planifier, d'organiser, d'exécuter et de suivre les tests manuels et automatisés, tout en permettant aux membres de l'équipe de communiquer et de collaborer efficacement. Cette plateforme offre une gamme de fonctionnalités telles que la création de cas de test, la gestion des suites de test, l'exécution des tests manuels et automatisés, la génération de rapports, la traçabilité des problèmes, la gestion des utilisateurs et des rôles, la collaboration en temps réel et bien plus encore. Les utilisateurs peuvent également intégrer Test Collab avec d'autres outils de développement, tels que des outils de gestion de projets, des outils de collaboration et des outils de gestion de code source (voir Figure 4).

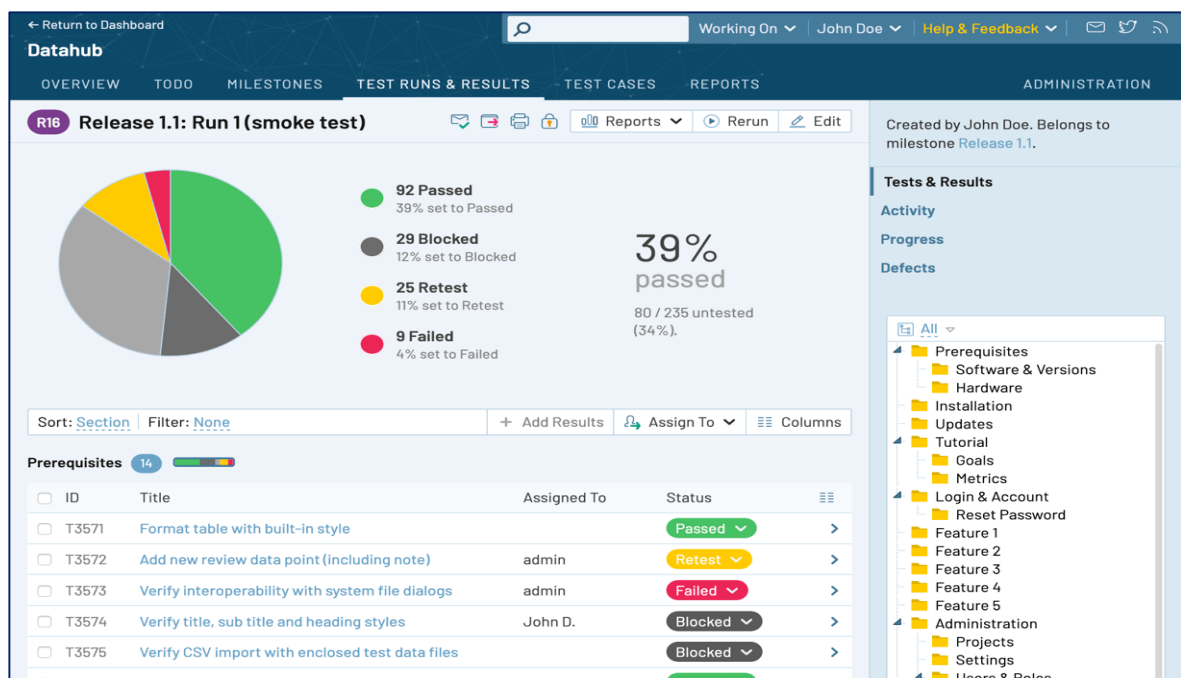


Figure 2: Aperçu sur TestRail

³ <https://testcollab.com/>

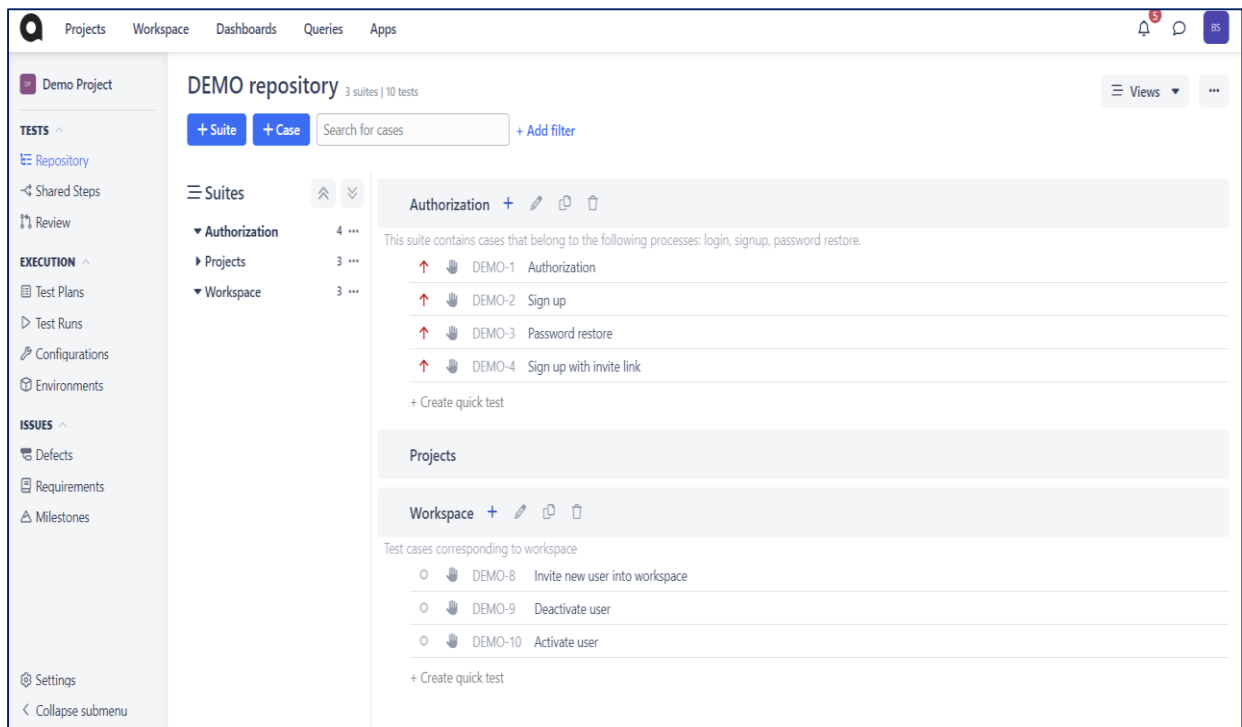


Figure 3: Aperçu sur Qase

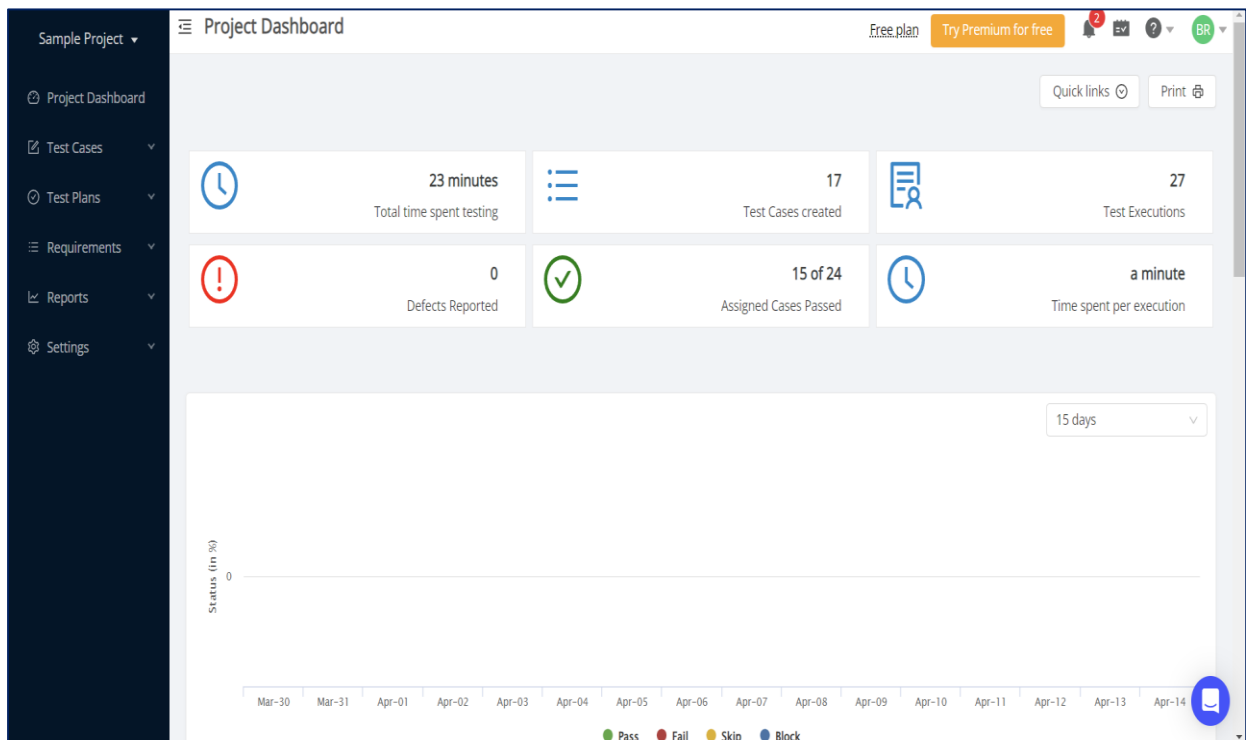


Figure 4: Aperçu sur Test Collab

Le tableau décrit la comparaison entre les logiciels présentés précédemment. Cette comparaison se base sur un ensemble de critères selon les fonctionnalités des tests logiciels et sur leur utilisation.

Table 1 : Comparaison entre les logiciels

	TestRail	Qase	TestCollab
Installation facile	✓	✓	✓
Création et maintenance des projets	✓	✓	✗
Utilisateur et création du rôle d'utilisateur	✓	✓	✓
Gestion des exigences	✓	✓?	✓?
Création des sprints de test	✗	✗	✗
Création des plans de tests	✓	✓	✓
Création des cas de tests	✓	✓	✓
Exécution des cas de tests	✓	✓	✓
Système de reportage	✓	✓	✓?
Système de suivi des défauts	✓	✓	✓
Système d'importation et exportation	✓	✓	✗
Intégration avec d'autres outils	✓	✓	✗

✓ critère existant gratuitement ✗ critère non existant ✓? critère existant mais payant

Après avoir analysé ce tableau, nous avons observé que la majorité des outils disponibles sont payants, mais ils offrent une installation facile. De plus, ces outils ne permettent pas la création de sprints de test, mais plutôt la création de cas de tests. Cependant, nous avons constaté que ces outils présentent certaines limitations. Ils ne permettent pas la création de sprints. En se basant sur cette analyse de l'existant et en tenant compte des exigences de notre entreprise, nous proposons de développer une application de gestion de tests fonctionnels.

Cette nouvelle application répondra à nos besoins spécifiques en offrant des fonctionnalités avancées pour la gestion des tests fonctionnels. Nous pourrons ainsi créer des sprints de test, gérer les cas de tests de manière efficace et répondre aux exigences de notre entreprise de manière optimale.

5 Solution proposée

Notre application permet d'assurer les objectifs fonctionnels suivants :

- ✓ **Authentification** : permettant de créer les comptes utilisateurs pour accéder aux fonctionnalités de l'application.
- ✓ **Gérer les utilisateurs** : le système doit permettre de gérer les utilisateurs.

- ✓ **Gérer les projets de test** : le système doit permettre de gérer les projets de test.
- ✓ **Gérer les sprints** : le système doit permettre de gérer des sprints.
- ✓ **Gérer les uses case** : le système doit permettre de gérer les uses case.
- ✓ **Gérer les cas de test** : le système doit permettre de gérer les cas de test.

En plus, nous essayer de respecter les objectifs non fonctionnels suivants :

- ✓ **La sécurité** : l'utilisateur doit entrer son login et mot de passe pour accéder à l'application.
- ✓ **La compatibilité** : L'application doit être compatible avec tous les systèmes d'exploitation, tous les navigateurs et tous les supports desktop ou mobile.
- ✓ **L'ergonomie** : L'application doit être simple et facile à manipuler par l'utilisateur.
- ✓ **La gestion des erreurs** : L'application doit gérer mieux les exceptions par l'affichage d'un message d'alerte ou d'erreur personnalisé.
- ✓ **La rapidité de traitement** : La durée d'exécution des traitements s'approche le plus possible du temps réel.

6 Méthodologie de développement : méthodologie Agile

Actuellement, la méthode Agile est largement adoptée pour la gestion de projets. Dans le cadre de notre projet, nous avons choisi d'opter pour cette méthode en particulier. La méthode Agile nous permet de modéliser les différents modules de manière progressive et itérative en utilisant le langage UML. Dans la section suivante, nous détaillons les raisons qui ont motivé notre choix pour cette méthode de développement.

6.1 Présentation des méthodes agiles

L'agilité répond principalement à la nécessité d'adapter les organisations à des environnements concurrentiels de plus en plus étendus et exigeants. Cette approche permet d'insuffler de la réactivité et de la performance dans les structures. Pour ce faire, les méthodes agiles permettent de gérer les projets de manière itérative et incrémentale afin de réduire le cycle de vie du logiciel et de répondre efficacement aux besoins du client. Ces méthodes se caractérisent par l'utilisation de cycles courts qui permettent de découper le projet en petits blocs et de les hiérarchiser en fonction des besoins. Parmi les principales méthodes agiles, on peut citer SCRUM, Agile Unified Process (Agile UP ou AUP), EXtreme Programming (XP), Rapid Application Development (RAD) et Crystal Clear. Notre société utilise la méthode SCRUM pour

gérer ses projets. Pour ces raisons, nous proposons d'appliquer les l'essentiel de la méthode SCRUM pour développer notre application.

6.2 Méthode Scrum

Scrum est l'infrastructure agile la plus utilisée. A l'instar d'autres méthodes agiles, Scrum est une démarche de gestion de projet qui fait du client (ou utilisateur) le principal pilote de l'équipe en charge des développements. Historiquement, elle est principalement mise en œuvre dans le domaine informatique, et dans celui du développement d'applications en particulier. Mais, elle est aussi utilisée de plus en plus dans d'autres domaines de l'ingénierie produit [4] (voir Figure 5 [5]).



Figure 5: Processus de SCRUM

En Scrum, l'équipe de développement est autoorganisée et doit être capable de reprendre les tâches d'un autre membre :

- ✓ **Le Product Owner** est responsable de la vision du produit à réaliser
- ✓ **Le Scrum Master** assume plusieurs rôles, notamment celui de coach, chargé de projet, développeur et animateur
- ✓ **Le Scrum Master** est chargé de faire respecter les cérémonies Scrum au sein de l'équipe et de s'assurer que les membres ne sont pas perturbés par des événements extérieurs, tout en assurant le reporting
- ✓ **Le Product Backlog** est un référentiel des exigences et des fonctionnalités
- ✓ **Le Sprint Backlog** est une liste de tâches identifiées par l'équipe à remplir pendant un sprint.

7 Tableau Backlog du product

Le Backlog est un artéfact très important dans Scrum. C'est l'ensemble des caractéristiques fonctionnelles ou techniques qui constituent le produit souhaité. Nous commençons par présenter l'ensemble des acteurs de notre projet.

7.1 Identification des acteurs

Un acteur présente une personne, une entité ou un système externe agissant d'une façon directe sur l'application. Il peut déclencher, arrêter ou planifier une fonctionnalité. Dans le cas de notre projet nous présentons les acteurs suivants :

Table 2 : Identification des acteurs

<i>Administrateur</i>	<i>Testeur</i>	<i>Visiteur</i>
		
Il peut manipuler toutes les fonctionnalités de l'application, il a comme mission principale de gérer la sécurité du système, son paramétrage et la répartition des rôles	Cet acteur est chargé de gérer plusieurs tâches à savoir la gestion des projets de test, des sprints et aussi la gestion des cas de tests.	Cet acteur est chargé de consulter les projets de test, des sprints et aussi les cas de tests. Il peut être un développeur (membre d'équipe Scrum).

7.2 Fonctionnalités du tableau backlog product

Dans le tableau suivant, nous présentons notre tableau de backlog product. Pour notre projet, c'est le product owner qui a proposé ces différentes fonctionnalités.

Table 3: 'Backlog du projet'

Id	Fonctionnalités	En tant que	User Story	Priorité
1	Authentification	Utilisateur	En tant qu'utilisateur il peut s'authentifier afin d'accéder à l'application.	Élevée
2	Gestion des utilisateurs	Administrateur	En tant qu'administrateur il peut afficher la liste des utilisateurs.	Moyenne
			En tant qu'administrateur il peut ajouter un utilisateur.	Moyenne
			En tant qu'administrateur il peut modifier un utilisateur.	Moyenne
			En tant qu'administrateur il peut supprimer un utilisateur.	Faible
3	Gestion de projets de test	Utilisateur	En tant qu'utilisateur il peut afficher la liste des projets de test.	Moyenne
		Testeur	En tant que testeur il peut ajouter des projets de test.	Elevée
			En tant que testeur il peut modifier des projets de test.	Moyenne
			En tant que testeur il peut supprimer des projets de test.	Faible
4	Gestion d'affectation des testeurs	Administrateur	En tant qu'administrateur il peut Afficher la liste des testeurs	Moyenne
			En tant qu'administrateur il peut affecter un testeur	Elevée

			En tant qu'administrateur il peut modifier l'affectation d'un testeur	Moyenne
			En tant qu'administrateur il peut supprimer l'affectation d'un testeur	Faible
4	Gestion des sprints	Utilisateur	En tant qu'utilisateur il peut afficher la liste des sprints.	Moyenne
		Testeur	En tant que testeur il peut ajouter des sprints.	Elevée
			En tant que testeur il peut modifier des sprints.	Moyenne
			En tant que testeur il peut supprimer des sprints.	Faible
5	Gestion des cas d'utilisation	Utilisateur	En tant qu'utilisateur il peut afficher la liste des cas d'utilisation.	Moyenne
		Testeur	En tant que testeur il peut ajouter des cas d'utilisation.	Elevée
			En tant que testeur il peut modifier des cas d'utilisation.	Moyenne
			En tant que testeur il peut supprimer des cas d'utilisation.	Faible
6	Gestion des cas de test	Utilisateur	En tant qu'utilisateur il peut afficher la liste des cas de test.	Moyenne
		Testeur	En tant que testeur il peut ajouter des cas de test.	Moyenne

			En tant que testeur il peut modifier des cas de test.	Moyenne
			En tant que testeur il peut supprimer des cas de test.	Faible
7	Proposer un scénario	Utilisateur	En tant qu'utilisateur il peut afficher la liste des scénarios.	Moyenne
		Testeur	En tant que testeur il peut ajouter des scénarios.	Moyenne
			En tant que testeur il peut modifier des scénarios.	Moyenne
			En tant que testeur il peut supprimer des scénarios.	Faible
8	Exécution des scénarios	Testeur	En tant que testeur il peut exécuter un scénario.	Elevée
			En tant que testeur il peut exporter des rapports de test.	Elevée

En se basant sur le tableau du backlog product et en discutant avec les membres de l'équipe SCRUM, nous avons fixé les sprints suivants :

- ✓ Le premier sprint contient les fonctionnalités :
 - Gestion des utilisateurs
- ✓ Le deuxième sprint contient les fonctionnalités suivantes :
 - Gestion des projets
 - Gestion d'affectation des testeurs
 - Gestion des sprints
 - Gestion des cas d'utilisation
 - Gestion des cas de test
- ✓ Le troisième sprint contient les fonctionnalités suivantes :
 - Gestion des scénarios
 - Exécution des scénarios

7.3 Planification du projet

Nous concluons par la planification. Voici le planning que nous avons suivi pour gérer ce projet durant la période de stage :

Table 4 : Planification de durées

Sprint	Fonctionnalité	Durée
Sprint 0 : Formation et environnement de travail	Formation	Du 1 au 20 Février
Sprint 1 : Gestion des utilisateurs	Gestion des utilisateurs	Du 21 Février au 17 Mars
Sprint 2 : Préparation plan de test	Gestion des projets de test	Du 20 Mars au 31 Mars
	Gestion des sprints	Du 3 Avril au 14 avril
	Gestion des cas d'utilisation	Du 17 Avril au 27 avril
	Gestion des cas de test	Du 28 Avril au 5 Mai
Sprint 3 : Exécuter plan de test	Gérer des scénarios	Du 8 Mai au 19 Mai
	Exécution des scénarios	Du 22 au 26 Mai

8 Conclusion

Dans ce premier chapitre, nous avons présenté une description sommaire de notre projet effectué à l'organisme d'accueil SiFAST. Nous avons défini quelques concepts de base liés aux tests logiciels qui seront exploités dans les chapitres suivants. Ensuite, nous avons présenté le tableau du backlog product proposé par le product owner de la société. Ces fonctionnalités seront réparties en trois sprints développés dans les chapitres suivants.

CHAPITRE 2 SPRINT 1 - GESTION DES UTILISATEURS

1 Introduction

Dans ce chapitre, nous allons développer le premier sprint selon les fonctionnalités présentées dans le tableau backlog product. Nous allons élaborer les diagrammes UML représentant respectivement l'analyse et la conception de ce module. Enfin, nous allons présenter la réalisation et les interfaces correspondantes.

2 Analyse

Cette partie présente toutes les fonctionnalités essentielles pour la gestion des utilisateurs dans notre projet. Dans notre cas, l'administrateur est principal pour gérer les comptes utilisateurs. L'utilisateur, selon notre société, un testeur ou bien un visiteur. Le visiteur peut un membre de l'équipe SCRUM.

- ✓ Diagramme de cas d'utilisation de premier sprint

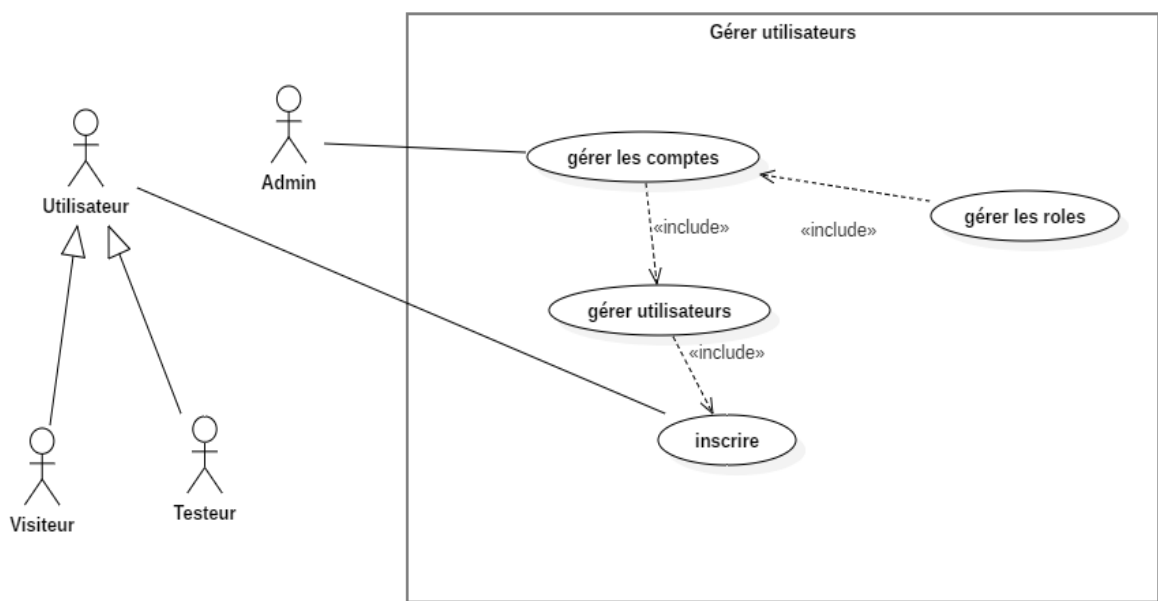


Figure 6: Diagramme de cas d'utilisation de premier sprint

Une fois le diagramme de cas d'utilisation élaboré, nous abordons la phase de conception où nous devons définir les diagrammes de séquence. Ces derniers permettent de décrire comment les différents éléments du système interagissent entre eux ainsi qu'avec les acteurs. Nous devons également créer le diagramme de classes pour définir la structure et les relations entre les différentes classes du système.

3 Conception

Cette section décrit les diagrammes de séquence et de classes modélisant les aspects dynamique et statique de notre système.

3.1 Diagramme de séquence du cas gérer compte

Nous allons modéliser le diagramme séquence du cas d'utilisation « **Gérer compte** » selon le scénario présenté par le tableau suivant.

Table 5 : Description de séquence (Gérer compte)

Cas d'utilisation	Gérer compte
Acteur	Admin
Précondition	l'admin est authentifié
Post condition	l'admin peut ajouter, modifier et supprimer les utilisateurs et leur compte
Description du scénario principal	• L'admin saisit son login et mot de passe pour accéder à l'interface, • L'admin accède à l'interface de gestion des utilisateurs. • Le système affiche le tableau qui contient tous les utilisateurs. • L'admin clique sur le bouton « Create », « Delete » ou bien « Update ». • Si l'admin veut ajouter un utilisateur, il va saisir ses informations, • Le système vérifie les informations, • L'admin confirme l'ajout, • L'admin crée le compte utilisateur, • Le système affiche « compte créé », • Si l'admin sélectionne le bouton modifier, il donner les informations à modifier, • Le système charge les informations des utilisateurs, • Le système affiche les informations.

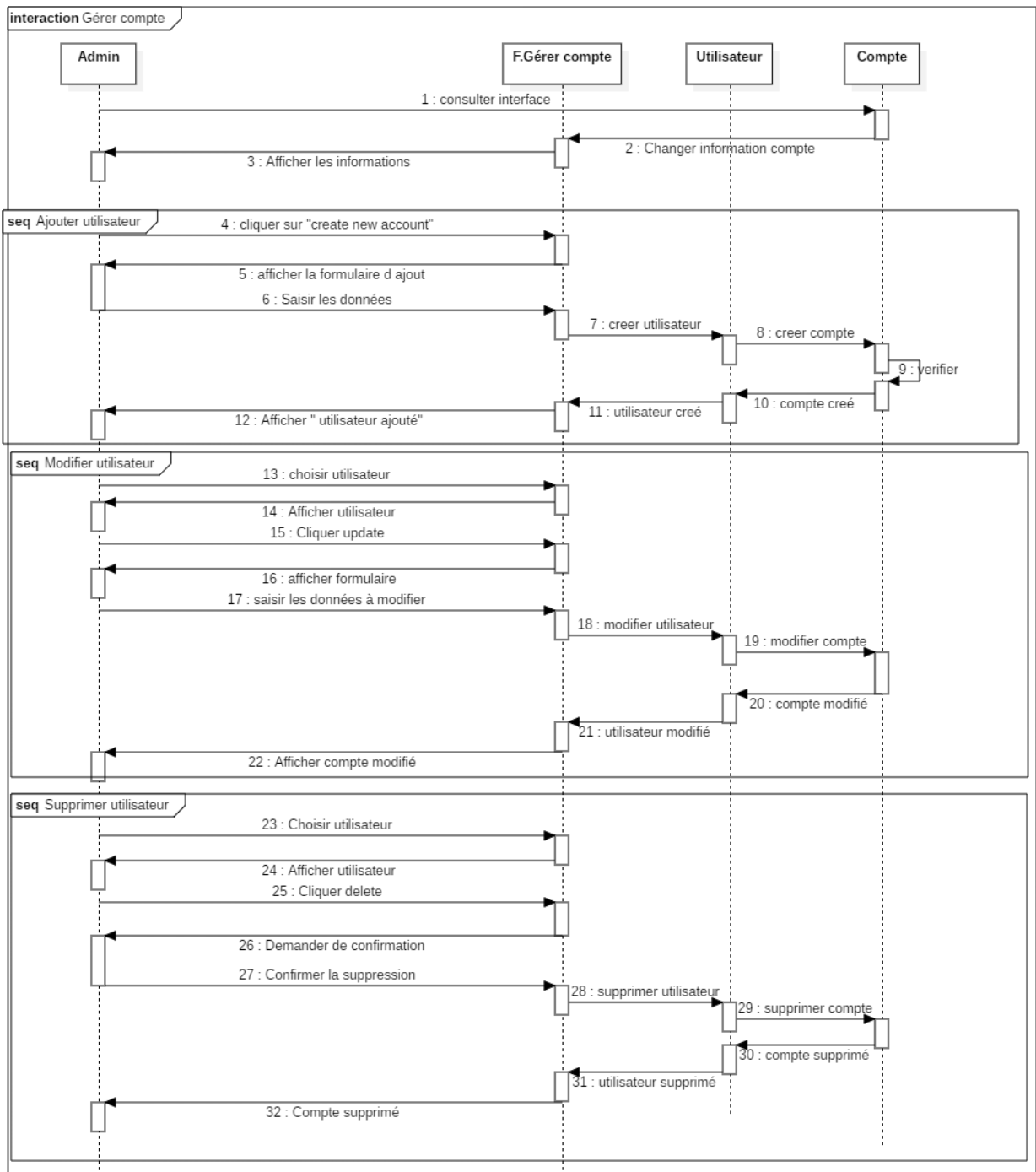


Figure 7: Diagramme de séquence « gestion de comptes utilisateurs »

3.2 Diagramme de séquence du cas d'utilisation « inscrire »

Nous allons modéliser le diagramme séquence du cas d'utilisation « inscrire » selon le scénario présenté par le tableau suivant.

Table 6: Scénario de la séquence (inscrire)

Cas d'utilisation	Authentifier
Acteur	Utilisateur
Précondition	L'utilisateur a installé l'application
Post condition	L'utilisateur peut s'inscrire
Description du scénario principal	• L'utilisateur clique sur le bouton « Register » • Le système affiche la page d'inscription. • L'utilisateur remplit les champs. • L'utilisateur clique sur le bouton « Register ». • le système enregistre les informations saisies dans base de données. • Le système affiche un toast « user Registered successfully ». • L'utilisateur saisit son login et mot de passe pour accéder à une interface.

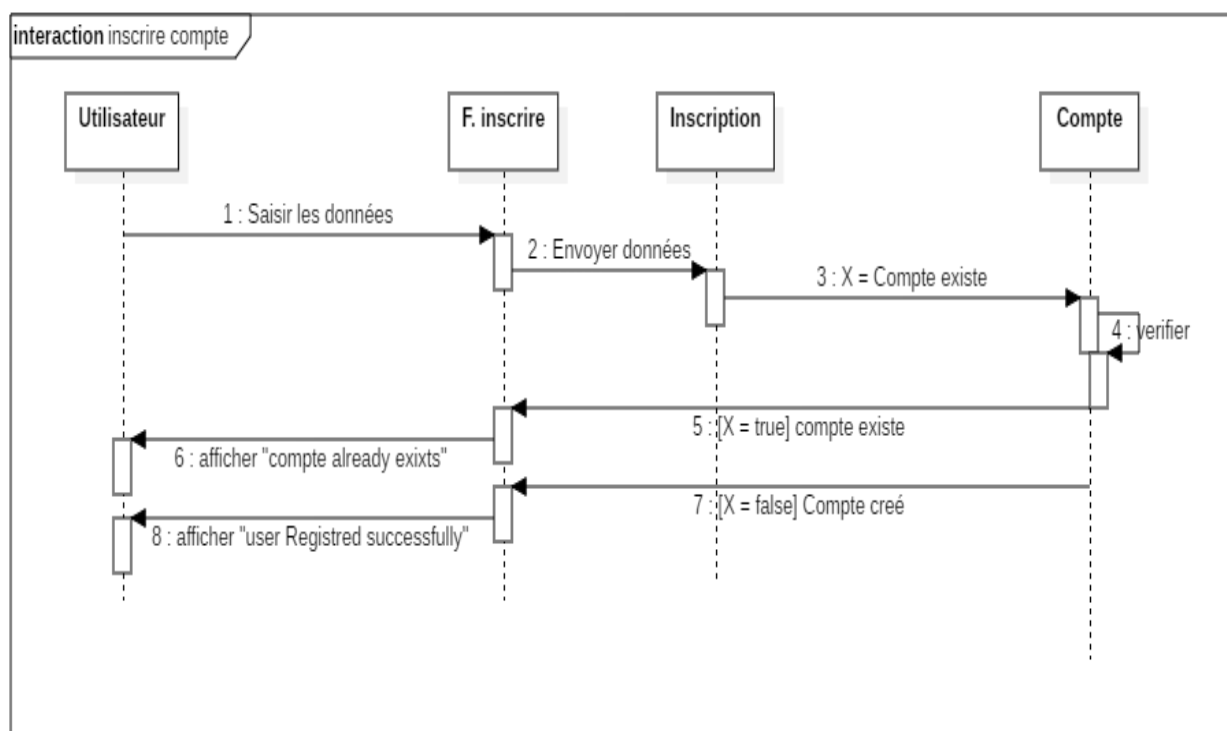


Figure 8: Diagramme de séquence « inscrire »

3.3 Diagramme de classes du premier sprint

Le tableau ci-dessous énumère l'ensemble des attributs associés à chaque classe, leurs types de données, ainsi que leurs brèves descriptions.

Table 7 : Description des classes (sprint1)

Classe	Attribut	Description de l'attribut	Type
Utilisateur	Id	Identifiant de l'utilisateur	Numérique
	Prénom	Prénom de l'utilisateur	Alphabétique
	Nom	Nom de l'utilisateur	Alphabétique
	Email	Adresse mail de l'utilisateur	Alphabétique
Rôle	Nom	Nom du rôle	Alphabétique
Testeur	Id_utilisateur	Rôle de l'utilisateur	Numérique
Visiteur	Id_utilisateur	Rôle de l'utilisateur	Numérique

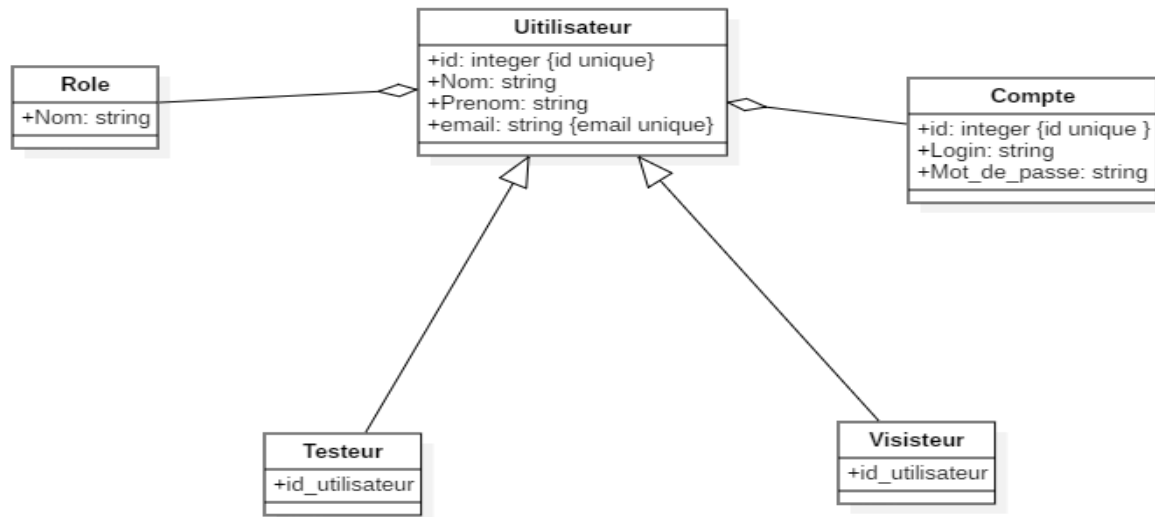


Figure 9: Diagramme de classe de premier sprint

4 Modèle relationnel

La transformation du diagramme de classe en modèle relationnel est assurée selon [6].

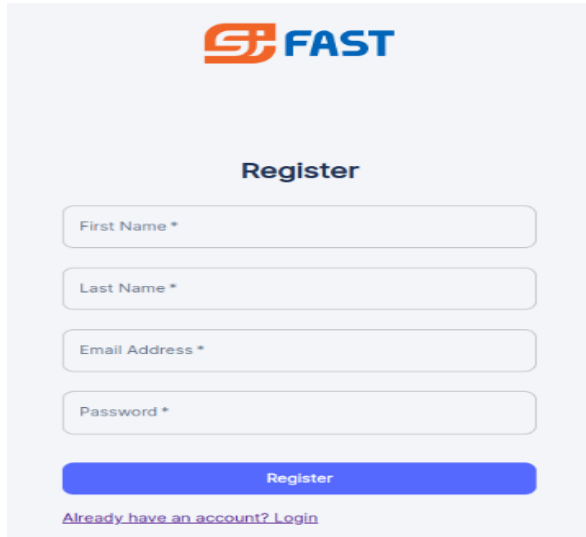
Utilisateur (Id, Prenom, Nom, Email).
Testeur (id, #id_utilisateur).
Visiteur (id, #id_utilisateur).
Compte (Id_compte, Login, Mot de passe).
Role (Id_Role, Nom).

5 Réalisation

Dans cette section, nous montrons les résultats de la réalisation du premier sprint sous forme des principales interfaces validées par les membres de l'équipe SCRUM.

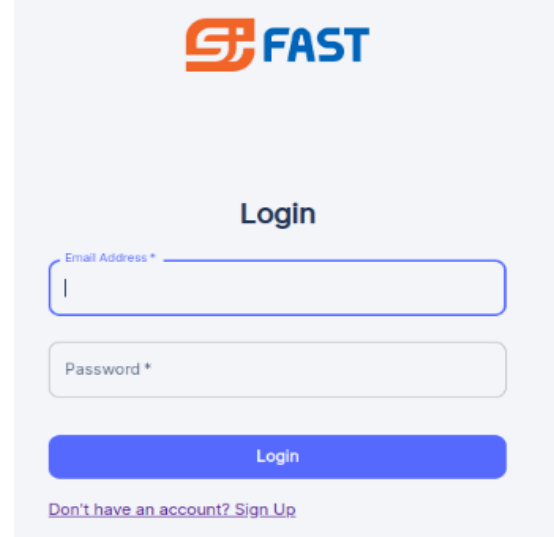
5.1 Interface de création d'un compte et interface de connexion

Les figures 11 et 12 représentent respectivement un formulaire d'inscription permettant à un utilisateur de saisir ses informations et une interface de connexion.



The Register form interface features the FAST logo at the top. Below it, the title "Register" is centered. The form contains four input fields: "First Name *", "Last Name *", "Email Address *", and "Password *". A blue "Register" button is positioned below the fields. At the bottom, there is a link: "Already have an account? Login".

Figure 10: Interface d'inscription

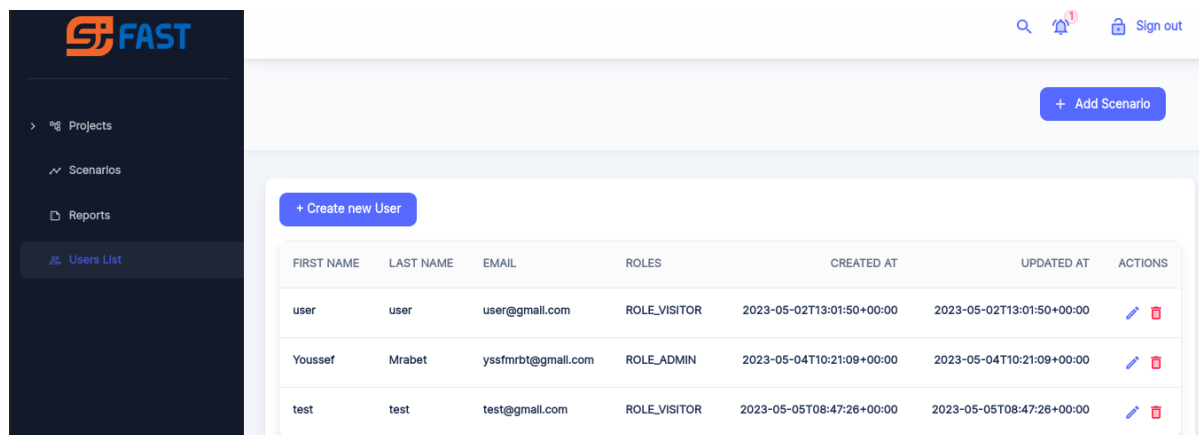


The Login form interface features the FAST logo at the top. Below it, the title "Login" is centered. The form contains two input fields: "Email Address *" and "Password *". A blue "Login" button is positioned below the fields. At the bottom, there is a link: "Don't have an account? Sign Up".

Figure 11: Interface de Login

5.2 Interface pour afficher la liste des utilisateurs

L'administrateur peut gérer les données d'un utilisateur en consultant l'interface suivante.



The Users List interface shows a sidebar with navigation options: Projects, Scenarios, Reports, and Users List (selected). The main content area has a "+ Add Scenario" button and a "+ Create new User" button. Below these is a table listing users.

FIRST NAME	LAST NAME	EMAIL	ROLES	CREATED AT	UPDATED AT	ACTIONS
user	user	user@gmail.com	ROLE_VISITOR	2023-05-02T13:01:50+00:00	2023-05-02T13:01:50+00:00	Edit Delete
Youssef	Mrabet	yssfmrbt@gmail.com	ROLE_ADMIN	2023-05-04T10:21:09+00:00	2023-05-04T10:21:09+00:00	Edit Delete
test	test	test@gmail.com	ROLE_VISITOR	2023-05-05T08:47:26+00:00	2023-05-05T08:47:26+00:00	Edit Delete

Figure 12: Interface affichant la liste des utilisateurs

6 Conclusion

Nous avons modélisé le premier sprint, la gestion des utilisateurs, en utilisant les diagrammes UML. Ensuite, nous avons montré les interfaces obtenues pour gérer leurs données. Ces données sont utiles pour développer les autres sprints.

CHAPITRE 3 SPRINT 2 - PREPARATION DU PLAN DE TEST

1 Introduction

Dans ce chapitre, nous traitons le deuxième sprint qui consiste à la gestion des projets de test, gestion des sprints, gestion des cas d'utilisation et gestion des cas de test pour avoir la démarche de test d'un projet. Ce sprint est essentiel pour préparer les éléments de base pour l'exécution des plans de tests.

2 Analyse

Dans cette section, nous décrivons l'ensemble des fonctionnalités nécessaires pour le sprint 2

✓ Diagramme de cas d'utilisation du deuxième sprint

La figure 13 représente les fonctionnalités du deuxième sprint en interaction avec le testeur. Le testeur peut ajouter des projets des tests selon leurs sprints, use cases et leurs cas de tests

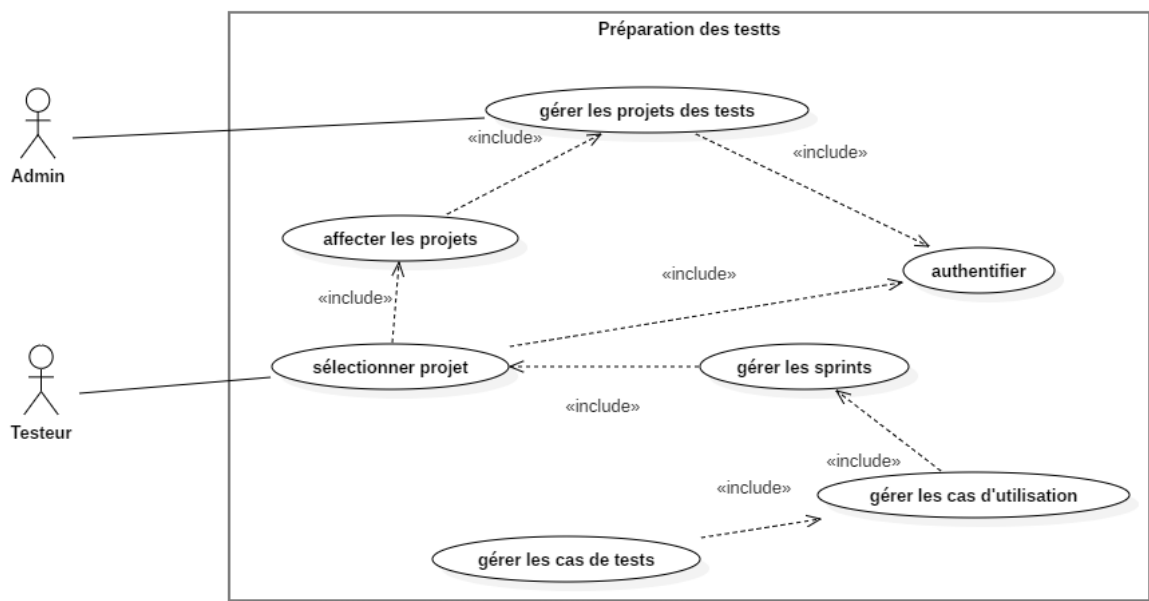


Figure 13: Diagramme de cas d'utilisation de sprint 2

3 Conception

Dans cette section, nous présentons les diagrammes de séquence des principaux cas d'utilisation du diagramme de la figure 13.

3.1 Diagrammes de séquence du cas d'utilisation « gérer les projets »

Nous allons modéliser le diagramme le plus important à savoir : gestion d'un projet de test et affectation des testeurs en suivant le scénario suivant.

Table 8 : Description de la séquence (Gérer projet de test)

Cas d'utilisation	Gérer projet
Acteur	Administrateur
Précondition	L'administrateur authentifié
Postcondition	L'administrateur peut ajouter, modifier, supprimer un projet de test
Description du scenario principal	• L'administrateur saisit son login et mot de passe pour accéder à l'interface. • L'administrateur accède à l'interface de gestion des projets de test • Le système affiche le tableau qui contient tous les projets de test. • L'administrateur clique sur le bouton « create new project » • Le système affiche l'interface d'ajout • L'administrateur peut affecter et désaffecter un testeur dans un projet de test • Le système affiche l'interface d'affectation et de désaffectation des testeurs • L'administrateur saisit les données d'un projet de test • L'administrateur clique sur le bouton « update » ou bien « delete » • L'administrateur peut consulter la liste des projets de test • Le système affiche la liste des projets de test

Nous allons modéliser le diagramme de séquence du cas d'utilisation « gérer les projets » (voir la figure 14).

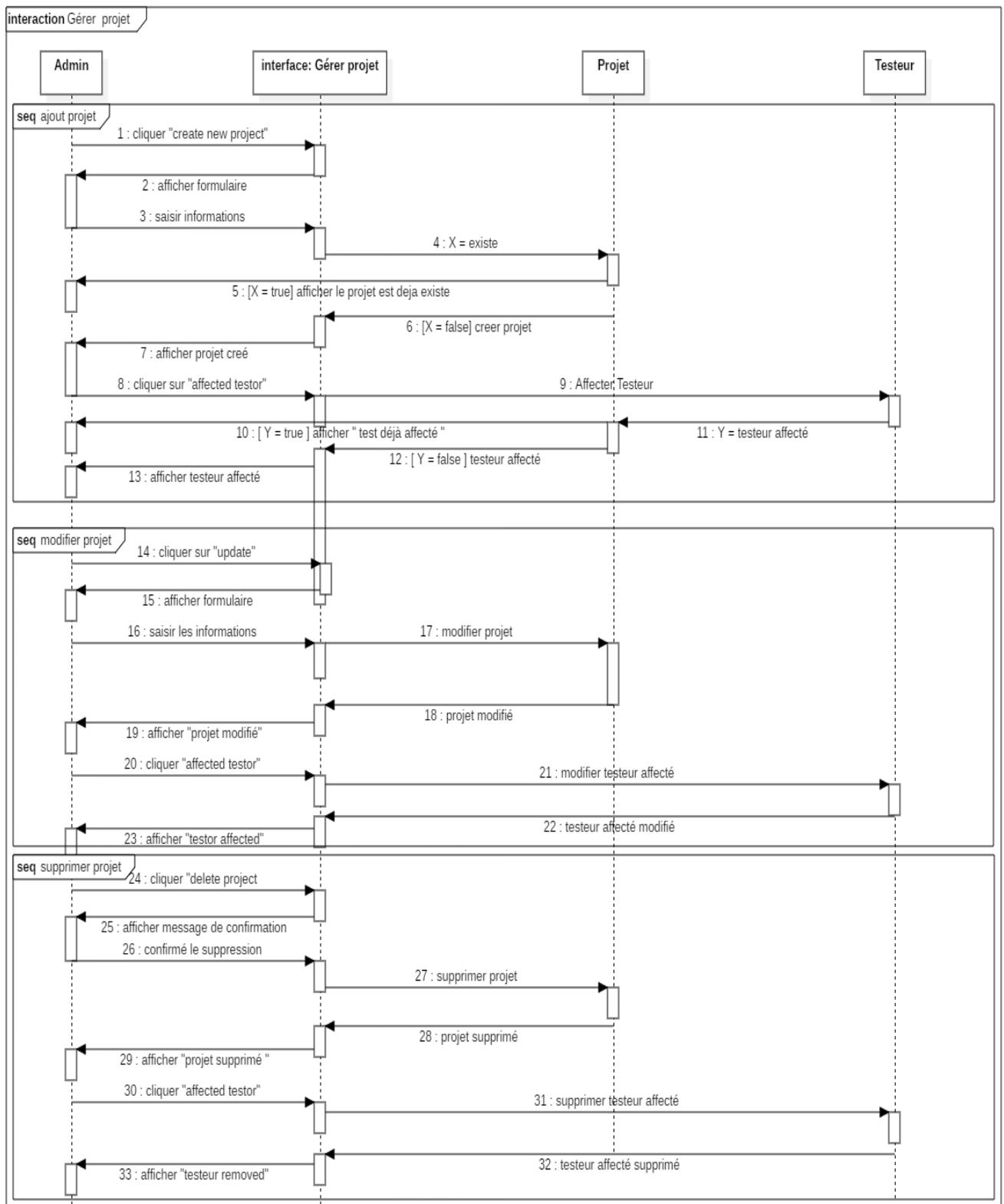


Figure 14: Diagramme de séquence du deuxième sprint « Gérer les projets »

3.2 Diagramme de séquence du cas d'utilisation « gérer les sprints »

Le diagramme séquence illustré dans la figure 15 modélise le cas d'utilisation « gérer les sprints » en suivant le scénario dans la Table 9.

Table 9 : Description de la séquence (gérer sprint)

Cas d'utilisation	Gérer les sprints
Acteur	Testeur
Précondition	Le testeur authentifié Le testeur affecté dans un projet
Postcondition	Le testeur peut ajouter, modifier, supprimer un sprint
Description du scenario principal	• Le testeur saisit son login et mot de passe pour accéder à l'interface. • Le testeur accède à l'interface de liste ses projets • Le testeur sélectionne le projet de test • Le système affiche le tableau qui contient tous les sprints de test. • Le testeur clique sur le bouton « create new sprint » • Le système affiche l'interface d'ajout • Le testeur saisie les données d'un sprint de test • Le testeur clique sur le bouton « update » ou bien « delete » • Le testeur peut consulter la liste des sprints de test • Le système affiche la liste des sprints de test

3.3 Diagramme de séquence du deuxième sprint « gérer les cas d'utilisation »

Le diagramme de séquence de la figure 16 modélise le scénario du cas d'utilisation « gérer un cas d'utilisation » selon la Table 10.

Table 10: Description de la séquence (Gérer cas d'utilisation)

Cas d'utilisation	Gérer les cas d'utilisation
Acteur	Testeur
Précondition	Le testeur authentifié
Postcondition	Le testeur peut ajouter, modifier, supprimer un cas d'utilisation
Description du scenario principal	• Le testeur saisit son login et mot de passe pour accéder à l'interface. • Le testeur sélectionne le sprint de test • Le testeur accède à l'interface de gestion des cas d'utilisation. • Le système affiche le tableau qui contient tous les cas d'utilisation. • Le testeur clique sur le bouton « create new use case » • Le système affiche l'interface d'ajout • Le testeur saisie les données d'un cas d'utilisation • Le testeur clique sur le bouton « update » ou bien « delete » • Le testeur peut consulter la liste des cas d'utilisation • Le système affiche la liste des cas d'utilisation

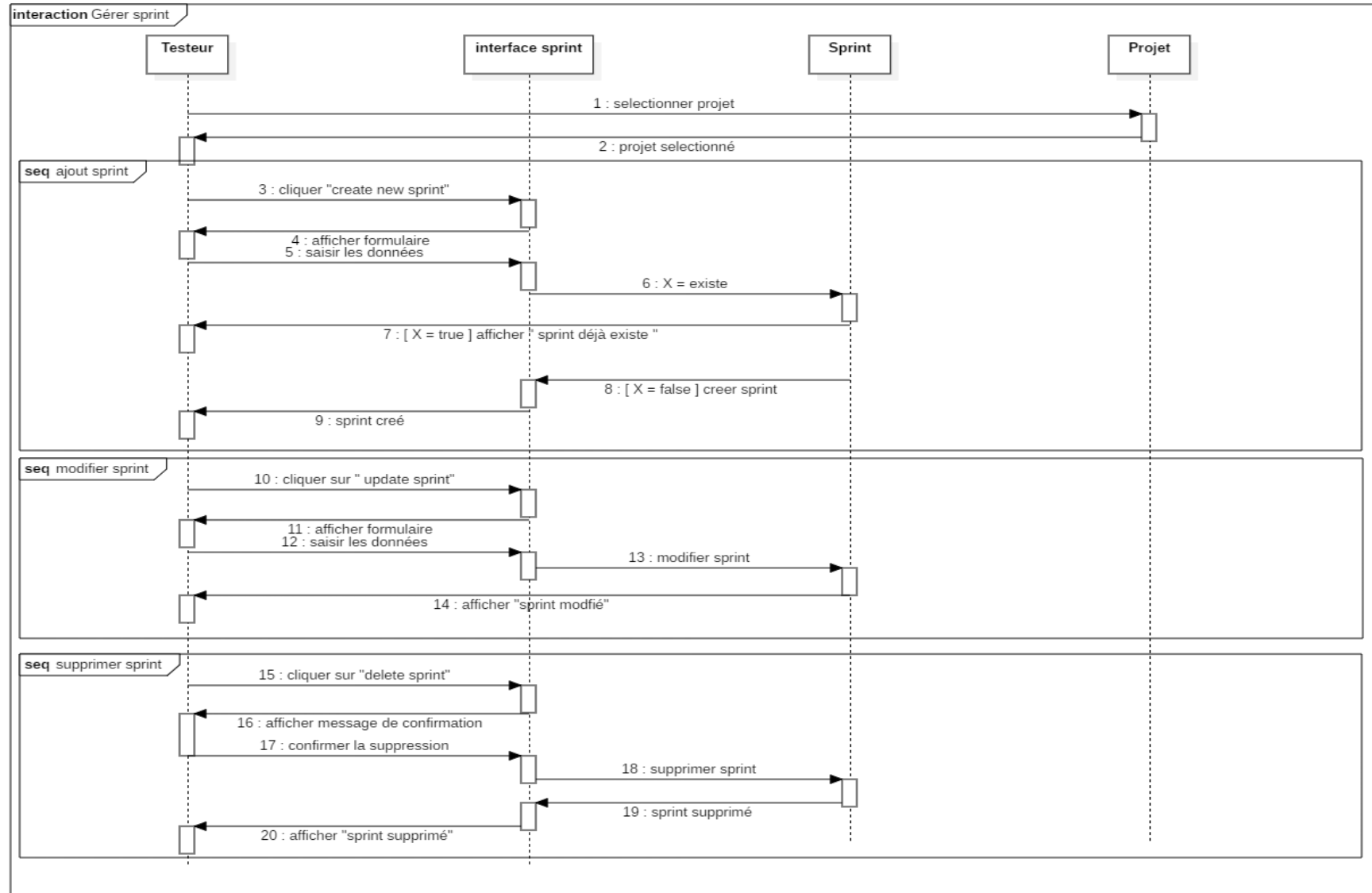


Figure 15: Diagramme de séquence du deuxième sprint « gérer les sprints »

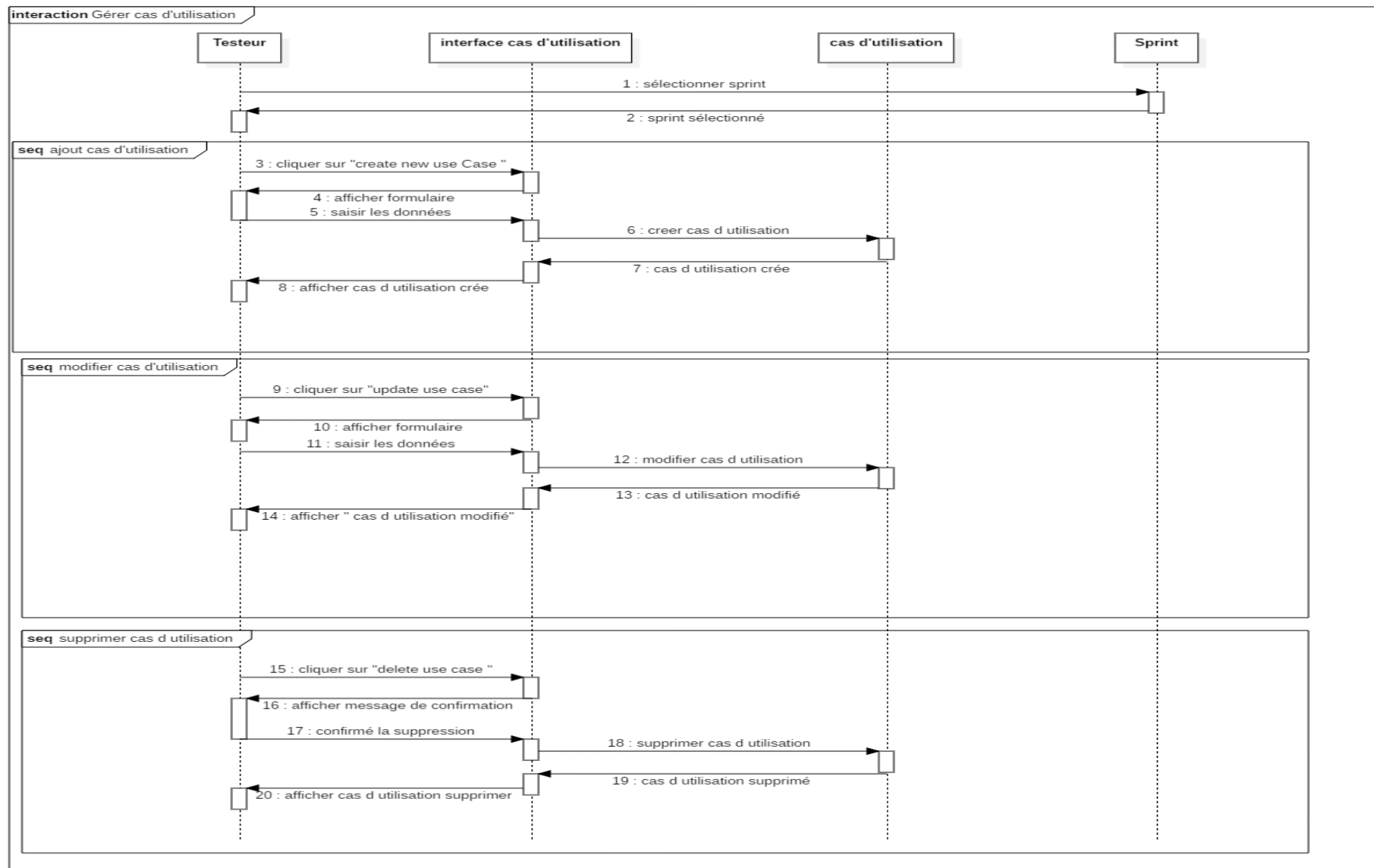


Figure 16: Diagramme de séquence du cas d'utilisation « gérer les cas d'utilisation »

3.4 Diagramme de séquence du deuxième sprint « gérer les cas de test »

Le diagramme de séquence de la figure 17 modélise le deuxième sprint « gérer les cas de test » en suivant le scénario de la Table 11.

Table 11 : Description de la séquence (Gérer cas de test)

Cas d'utilisation	Gérer les cas de test
Acteur	Testeur
Précondition	Le testeur authentifié
Postcondition	Le testeur peut ajouter, modifier, supprimer un cas de test
Description du scenario principal	- Le testeur saisit son login et mot de passe pour accéder à l'interface. - Le testeur sélectionne le cas d'utilisation - Le testeur accède à l'interface de gestion des cas de test. - Le système affiche le tableau qui contient tous les cas de test. - Le testeur clique sur le bouton « create new test case » - Le système affiche l'interface d'ajout - Le testeur saisit les données d'un cas de test - Le testeur clique sur le bouton « update » ou bien « delete » - Le testeur peut consulter la liste des cas de test - Le système affiche la liste des cas de test

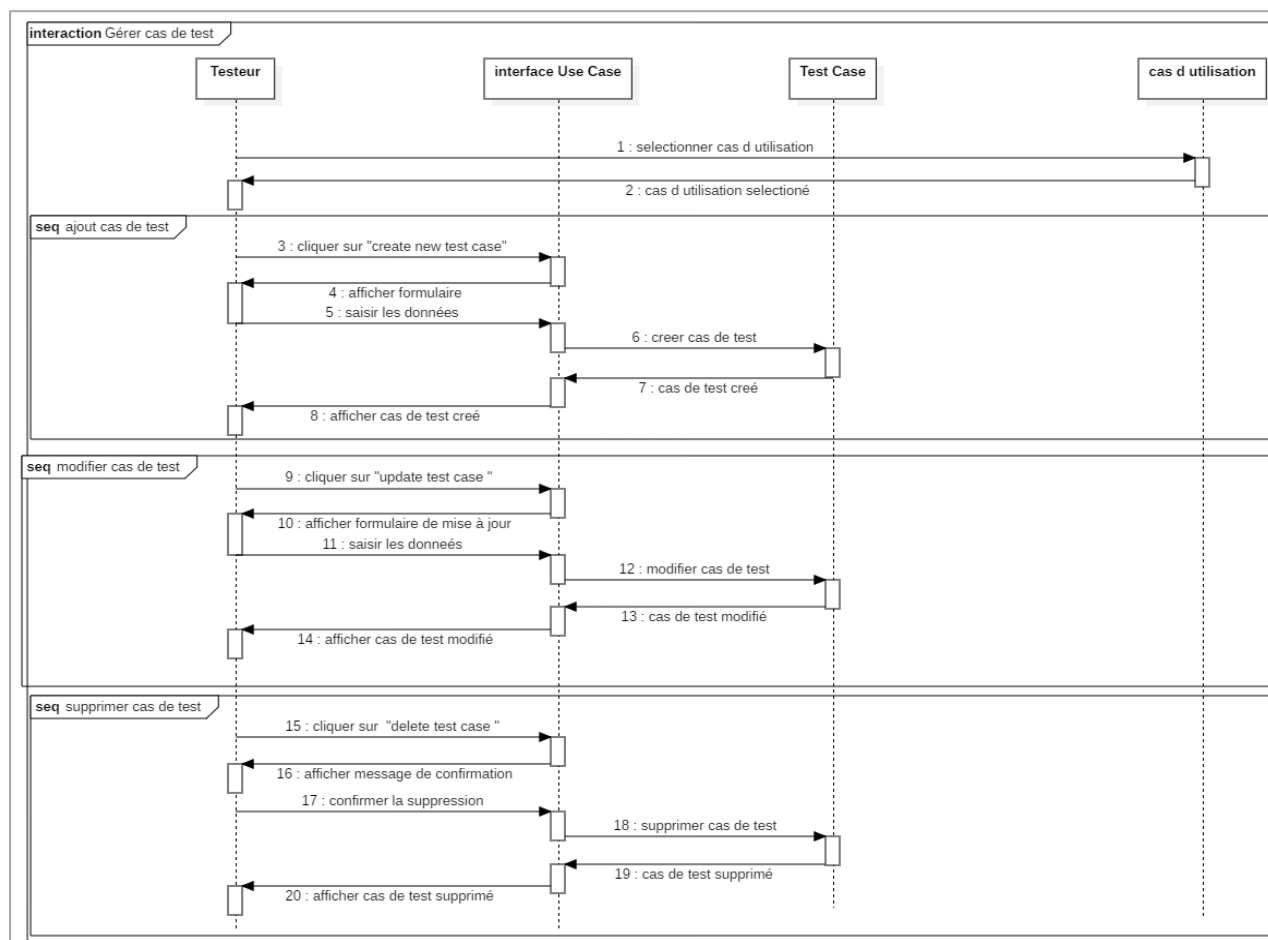


Figure 17: Diagramme de séquence du deuxième sprint « gérer cas de test »

3.5 Diagramme de classes

La Table 12 énumère l'ensemble des attributs associés à chaque classe, leurs types de données, ainsi que leurs brèves descriptions.

Table 12 : Description de classes (sprint 2)

Classe	Attribut	Description de l'attribut	Type
Testeur	Id_utilisateur	Identifiant d'utilisateur	Numérique
Affectation	Id_testeur	Identifiant testeur	Numérique
	Id_projet	Identifiant projet	Numérique
Projet de test	Id	Identifiant de projet	Numérique
	Nom	Nom du projet	Alphabétique
	Date de création	Date de création du projet	Date
	Auteur	Auteur de projet	Alphabétique
	Client	Client de projet	Alphabétique
Sprint	Id	Identifiant de Sprint	Numérique
	Nom	Nom de sprint	Alphabétique
	Priorité	Priorité de Sprint	Date
	Date début	Date de début de sprint	Date
	Date fin	Date de fin de sprint	Date
	User Story	User story de sprint	Alphabétique
Use Case	Id	Identifiant de use Case	Numérique
	Titre	Titre de use Case	Alphabétique
	Prérequis	Prérequis de use Case	Alphabétique
	Résultat attendu	Résultat attendu de use Case	Alphabétique
Cas de test	Id	Identifiant de cas de test	Numérique
	Nom	Nom de cas de test	Alphabétique
	Résumé	Résumé de cas de test	Alphabétique
	Titre	Titre de cas de test	Alphabétique
	Acteur	Acteur de cas de test	Alphabétique
	Précondition	Précondition de cas de test	Alphabétique

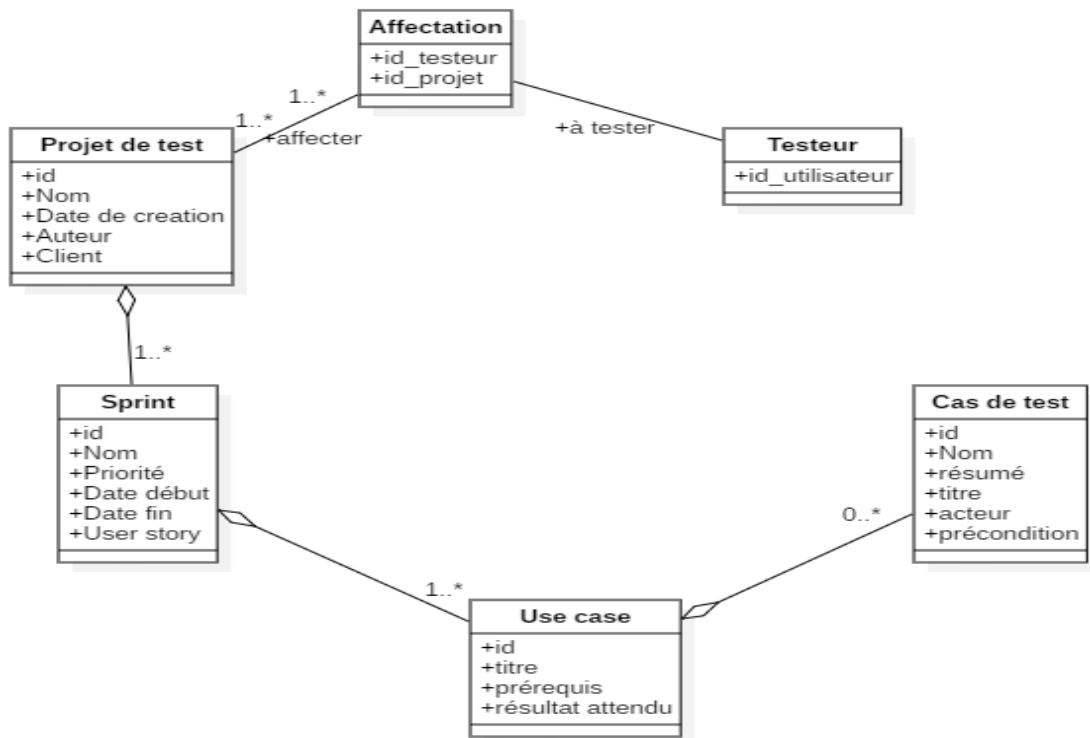


Figure 18: Diagramme de classe du deuxième sprint

4 Modèle relationnel

La transformation du diagramme de classe en modèle relationnel est représentée par les relations suivantes :

Testeur (*Id_utilisateur*)
Affectation (#*Id_testeur*, #*Id_projet*)
Projet_de_test (*Id*, *Nom*, *date_de_creation*, *Auteur*, *Client*).
Sprint (*id_Sprint*, *nom_sprint*, *priorite*, *date_debut*, *date_fin*, *user_story*, #*id_project*).
Use_Case (*Id_use_case*, *titre*, *prerequis*, *resultat_attendu*, #*id_sprint*).
Cas_de_test (*Id_cas_de_test*, *nom*, *resume*, *titre*, *acteur*, *precondition*, #*id_useCase*).

5 Réalisation

Cette section présente les interfaces pour préparer les plans de tests. Ces derniers se basent sur l'enchaînement entre projet de test, ses sprints, les cas d'utilisation et les cas du test.

5.1 Interfaces pour la gestion des projets de test

La figure 19 affiche la liste des projets de test. L'administrateur peut gérer les données d'un projet.

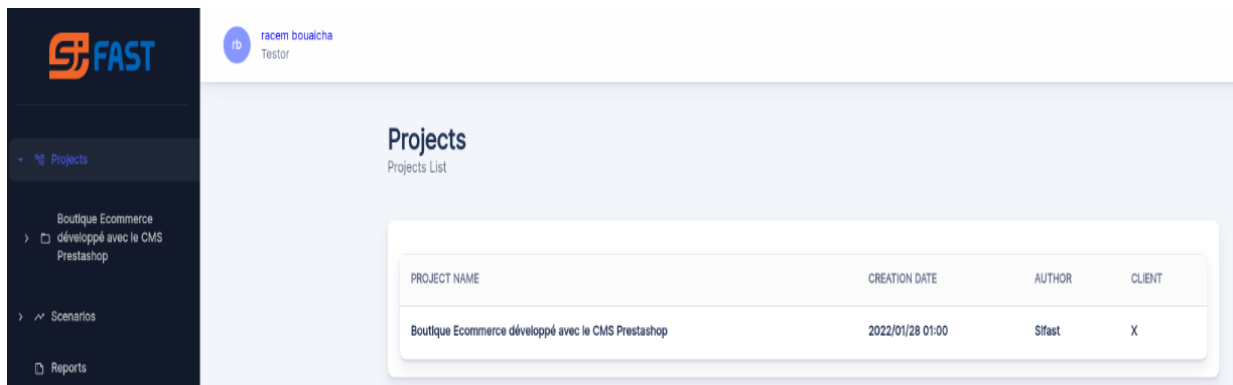


Figure 19: interface liste projet de test

En tant qu'administrateur lorsqu'on clique sur « create new project » on obtient ce formulaire suivant (voir figure 20). Cette interface permet de saisir les informations d'un projet (Nom du projet, auteur, date, etc.).

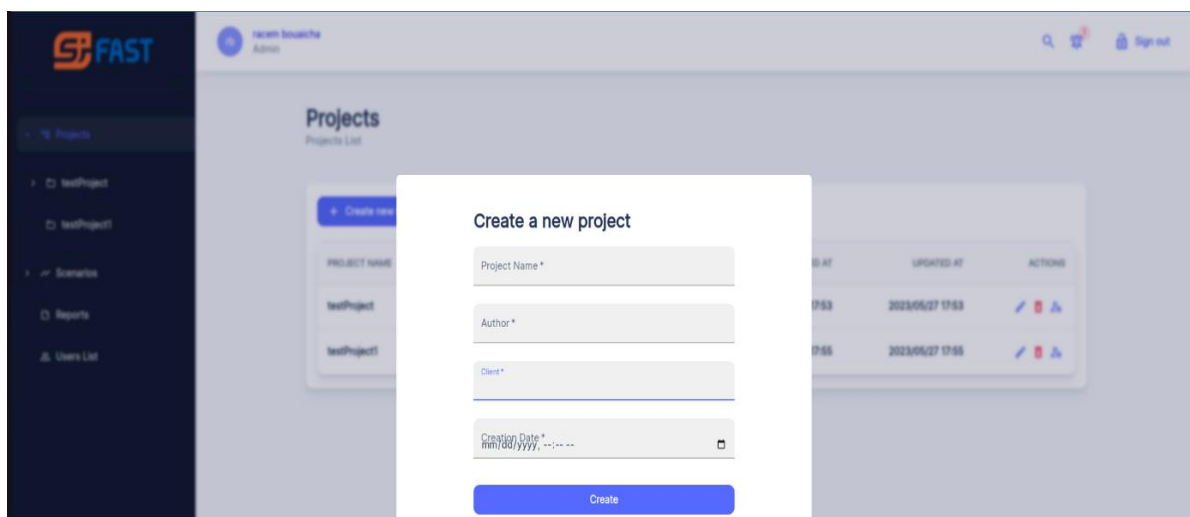


Figure 20: Interface de création projet de test

5.2 Interfaces pour la gestion des sprints de test

L'interface de la figure 21 affiche la liste des sprints de test. Le testeur peut gérer les données des sprints.

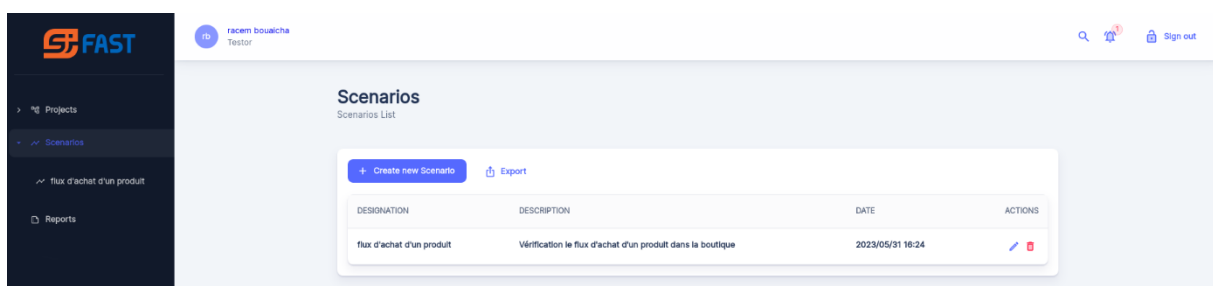


Figure 21: Interface liste des sprints de test

Le testeur peut cliquer sur « create new sprint » pour créer un sprint de test (voir figure 22).

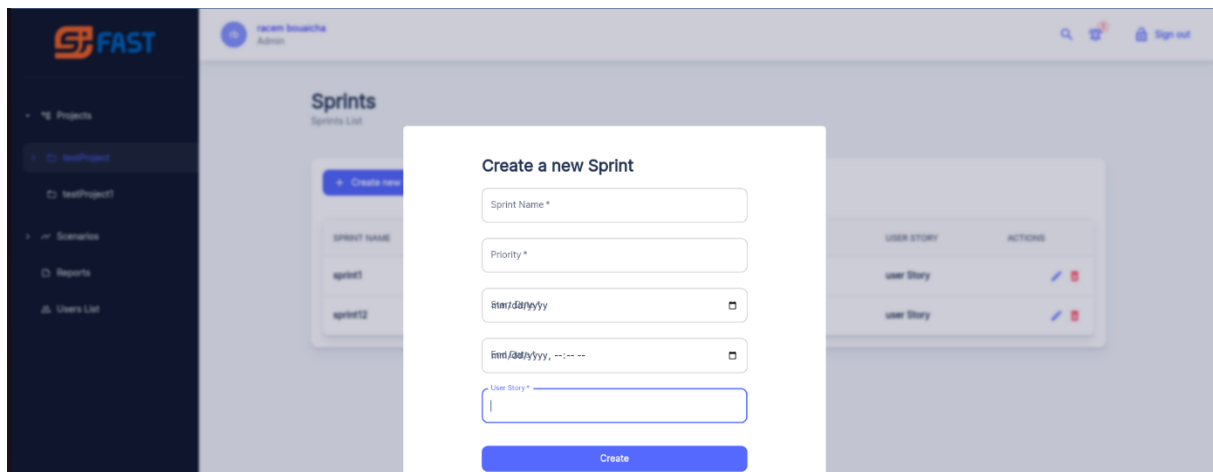


Figure 22: Interface de création de sprint

5.3 Interfaces pour la gestion des cas d'utilisation

La figure 23 représente une interface de l'affichage de la liste des cas d'utilisation du test.

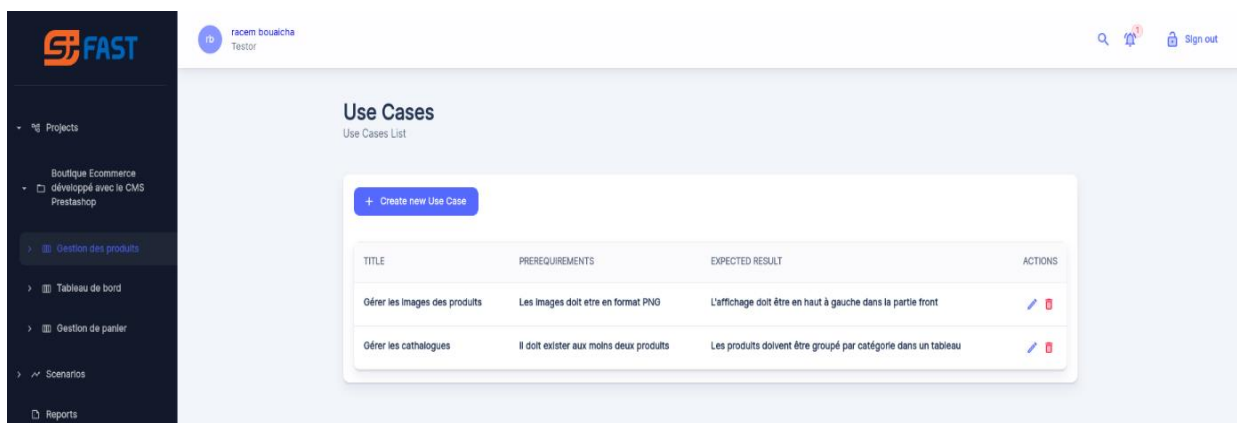


Figure 23: Interface affichage de la liste des cas d'utilisation

Le testeur peut ajouter un nouveau cas d'utilisation en cliquant sur « create new use case » selon l'interface suivante (voir figure 24).

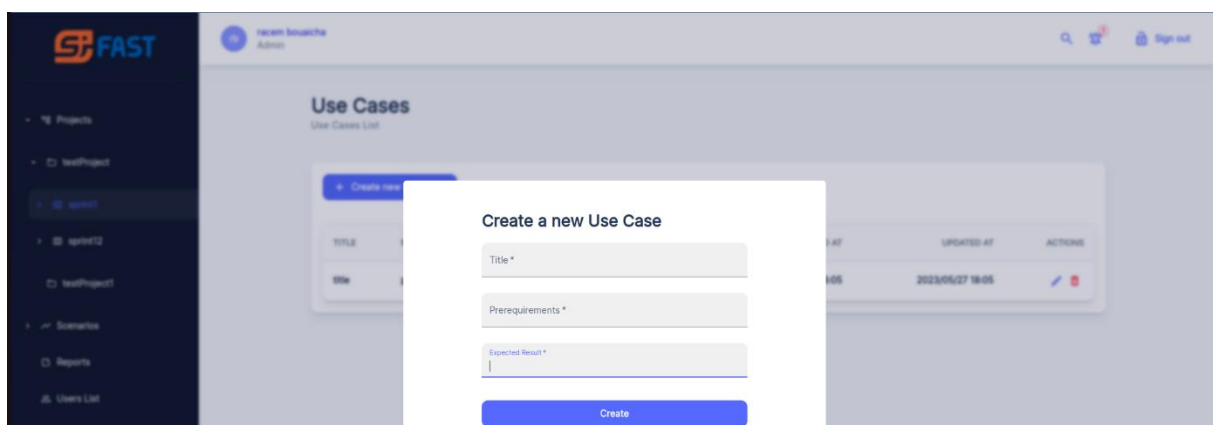


Figure 24: Interface de création de cas d'utilisation

5.4 Interfaces de gestion d'un cas de test

Cette interface, schématisée par la figure 25, affiche la liste des cas de tests.

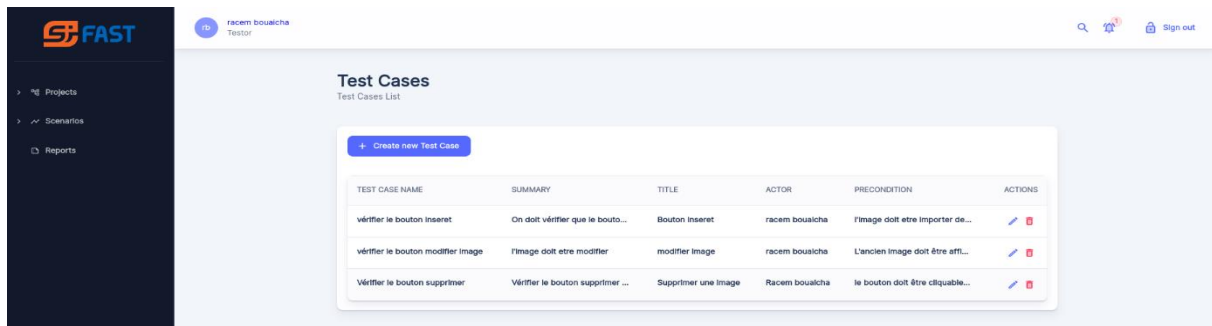


Figure 25: Interface liste des cas de tests

Le testeur peut cliquer sur « create new test case » afin d'ajouter un cas de test. Pour plus d'informations, voir la figure 26.

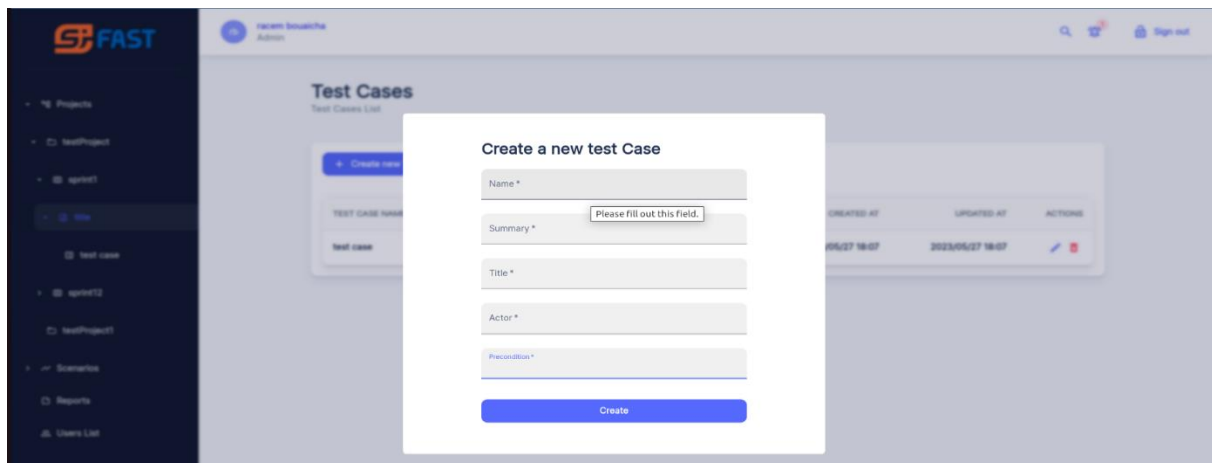


Figure 26: Interface de création de cas de test

6 Conclusion

Ce sprint est très important. Il permet de préparer les plans de test en déterminant les projets de test. Chaque projet va contenir des sprints de test. Chaque sprint peut contenir plusieurs cas d'utilisation de test. Chaque cas d'utilisation peut contenir des cas de tests. Ces données seront stockées et exploitées par le testeur.

CHAPITRE 4 SPRINT 3 - EXECUTION DU PLAN DE TEST

1 Introduction

Dans ce chapitre, nous allons développer le troisième sprint permettant l'exécution du plan du test proposé par le testeur. Nous allons exploiter les données d'un projet affecté par l'administrateur, obtenues au sprint 2. Ces données concernent les cas de tests appartenant aux cas d'utilisation d'un sprint.

2 Analyse

Dans cette section, nous décrivons l'ensemble des fonctionnalités nécessaires pour ce sprint.

- ✓ Diagramme de cas d'utilisation du troisième sprint

La figure 27 représente les fonctionnalités du troisième sprint en interaction avec le testeur. Le testeur peut proposer les scénarios de projet de test et les exécuter.

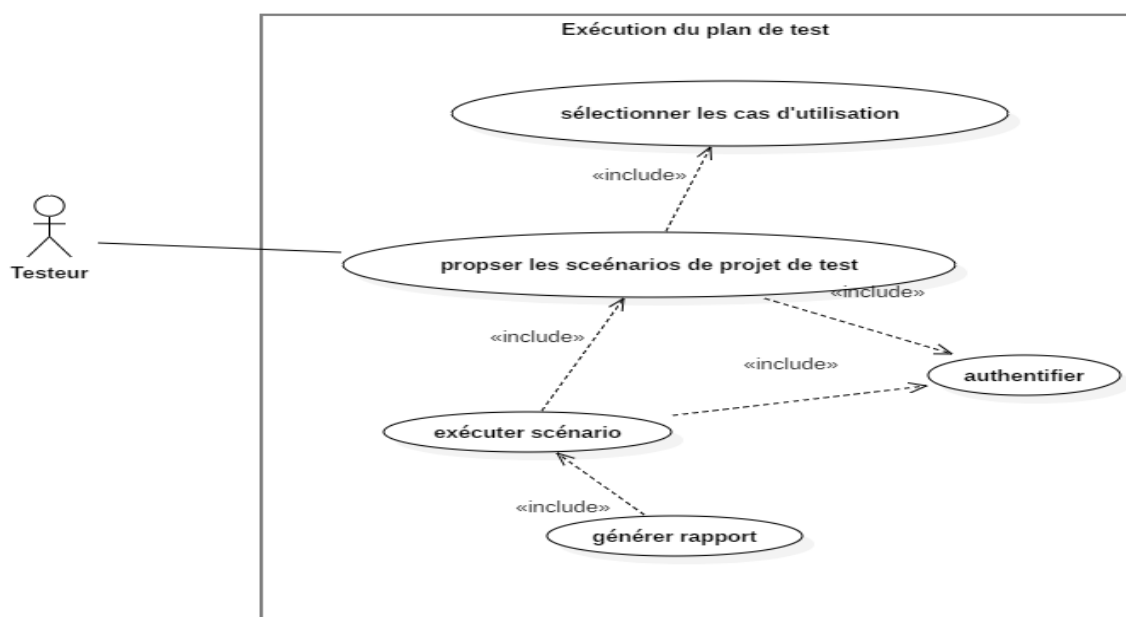


Figure 27: Diagramme de cas d'utilisation du troisième sprint

3 Conception

Dans cette section, nous allons mettre en évidence les fonctionnalités effectuées par le testeur. La première fonctionnalité concerne la proposition d'un scénario selon les données réalisées durant le deuxième sprint. Dans notre cas le scénario est composé d'un ensemble de cas d'utilisation appartenant à tous les projets à tester. Cette fonctionnalité par le cas d'utilisation suivant.

3.1 Diagramme de séquence du cas d'utilisation « proposer un scénario »

Nous allons modéliser le diagramme : proposition d'un scénario en suivant le scénario suivant (voir figure 28).

Table 13 : Description de la séquence (Gérer scénario)

Cas d'utilisation	Proposer un scénario
Acteur	Testeur
Précondition	Le testeur authentifié
Postcondition	Le testeur peut proposer un scénario
Description du scénario principal	• Le testeur saisit son login et mot de passe pour accéder à l'interface. • Le testeur accède à l'interface de liste des scénarios. • Le système affiche le tableau qui contient tous les scénarios. • Le testeur peut créer un scénario. • Le testeur peut ajouter un cas d'utilisation.

3.1 Diagramme de séquence du cas d'utilisation « exécuter un scénario »

La deuxième fonctionnalité concerne l'exécution du scénario sur un projet affecté. Comme résultat, nous obtenons un rapport contenant les états des cas de test. Cette fonctionnalité est modélisée par le diagramme de séquence de la figure 29.

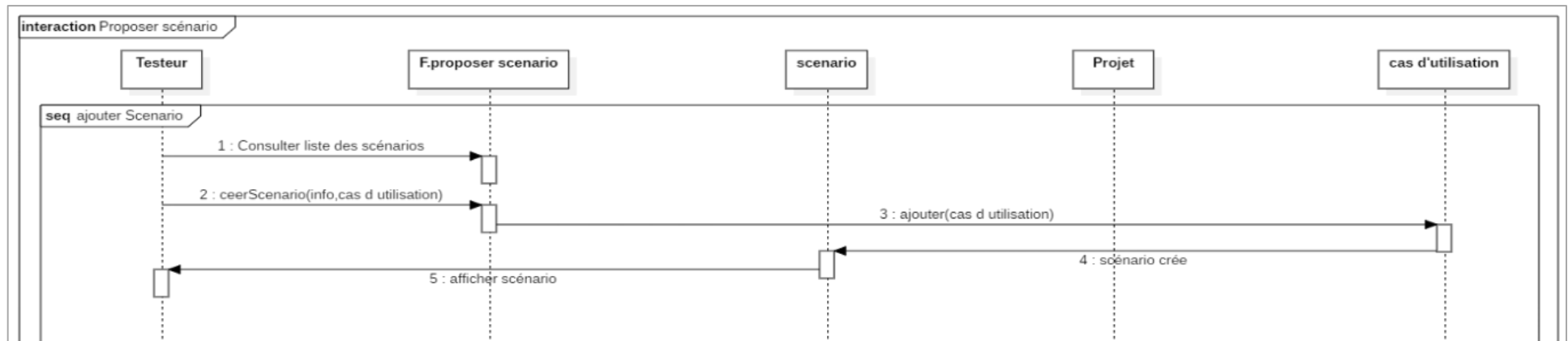


Figure 28: Diagramme de séquence du troisième sprint (Proposer scénario)

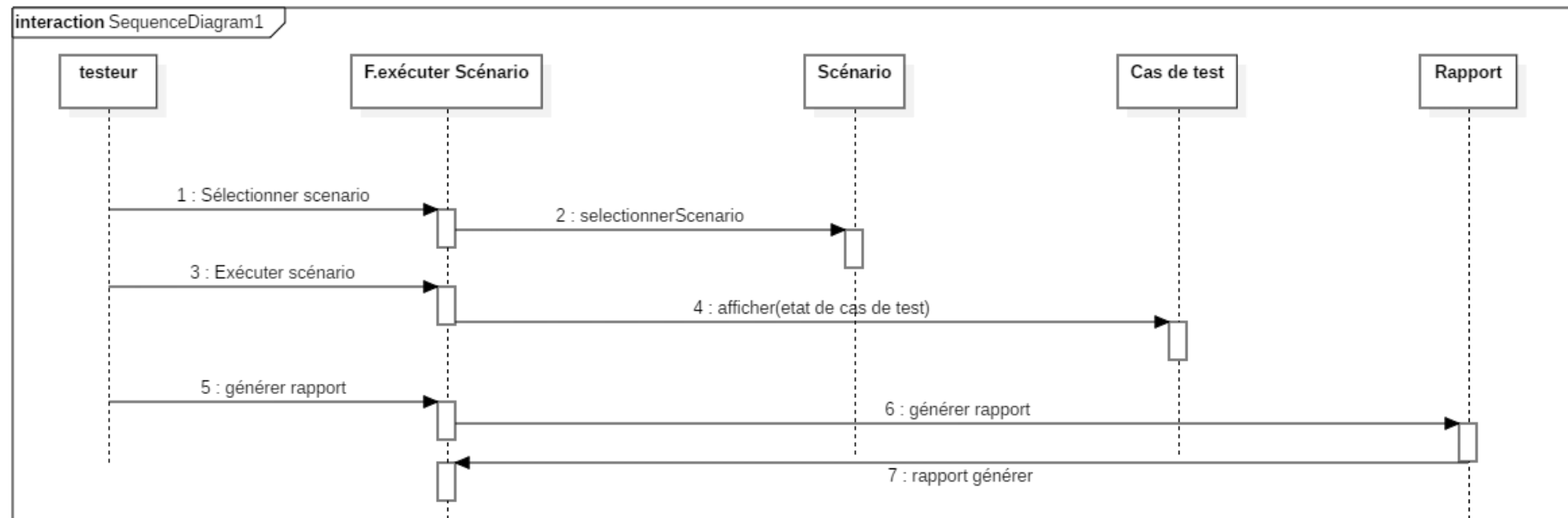


Figure 29: Diagramme de séquence du troisième sprint (Exécuter un scénario)

Table 14 : Description de la séquence (Gérer scénario)

Cas d'utilisation	Exécuter un scénario
Acteur	Testeur
Précondition	Le testeur est authentifié
Postcondition	Le testeur peut exécuter un cas de test
Description du scénario principal	• Le testeur saisit son login et mot de passe pour accéder à l'interface. • Le testeur accède à l'interface de gestion des scénarios. • Le système affiche le tableau qui contient tous les scénarios. • Le testeur sélectionne un scénario • Le testeur exécute un scénario • Le système change l'état de cas de test • Le testeur peut générer un rapport.

3.2 Diagramme de classe

La Table 15 énumère l'ensemble des attributs associés à chaque classe, leurs types de données, ainsi que leurs brèves descriptions.

Table 15 : Description de classe sprint 3

Classe	Attribut	Description de l'attribut	Type
Scénario	Id	Identifiant de Scénario	Numérique
	Désignation	Désignation de scénario	Alphabétique
	Description	Description de scénario	Alphabétique
	Date_creation	Date de création d'un scénario	Date
Rapport	Id	Identifiant de Rapport	Numérique
	Date_exportation	Date d'exportation d'un rapport	Date
Exécution	Id	Identifiant d'Exécution	Numérique
	Date_execution	Date d'exécution d'un cas de test	Date
	Support	Support de l'exécution	Alphabétique

	etat	État de l'exécution	Alphabétique
Cas d'utilisation	Id	Identifiant de use Case	Numérique
	Titre	Titre de use Case	Alphabétique
	Prérequis	Prérequis de use Case	Alphabétique
	Résultat attendu	Résultat attendu de use Case	Alphabétique
Cas de test	Id	Identifiant de cas de test	Numérique
	Nom	Nom de cas de test	Alphabétique
	Résumé	Résumé de cas de test	Alphabétique
	Titre	Titre de cas de test	Alphabétique
	Acteur	Acteur de cas de test	Alphabétique
	Précondition	Précondition de cas de test	Alphabétique

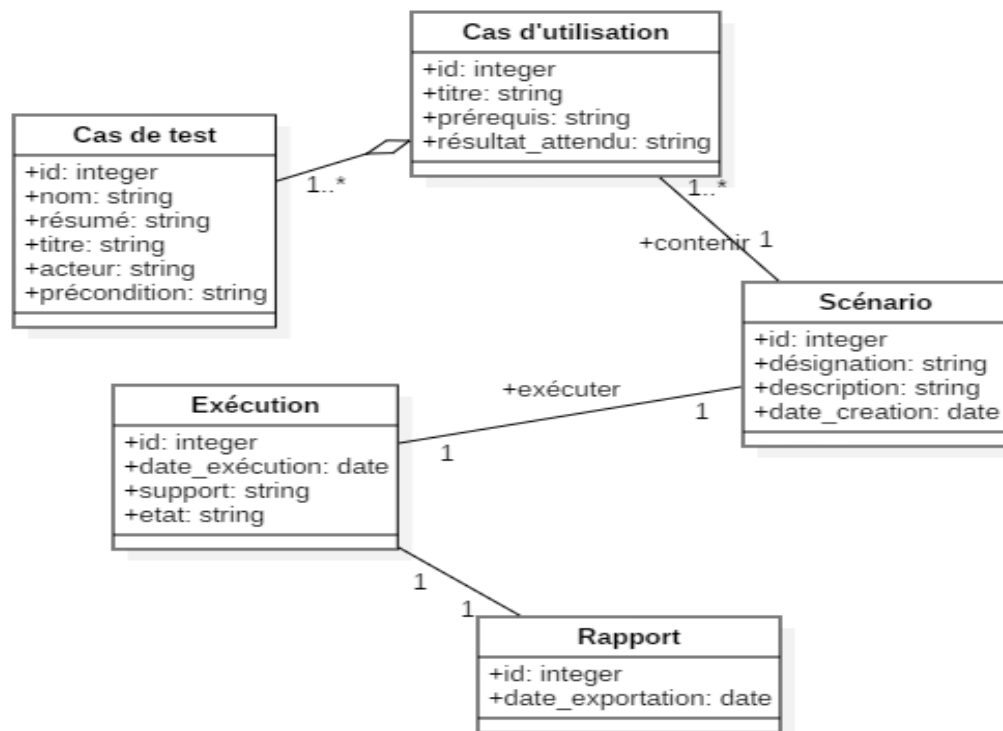


Figure 30: Diagramme de classe du troisième sprint

4 Modèle relationnel

La transformation du diagramme de classe en modèle relationnel

Scenario (Id, Désignation, Description, Date_creation, #id_execution).

Rapport (id, Date_exportation).

Execution (id, Date_execution, support, etat, #id_rapport, #id_scenario)

Use_Case (Id_use_case, titre, prerequis, resultat_attendu, #id_scenario).

Cas_de_test (Id_cas_de_test, nom, resume, titre, acteur, precondition, #id_useCase).

5 Réalisation

Dans cette section présente les interfaces pour exécuter les plans de tests. Elles se basent sur l'enchaînement entre scénarios, ses cas d'utilisation et ses cas de test.

5.1 Interfaces de liste des scénarios

Cette interface, schématisée par la figure suivante, affiche la liste des scénarios.

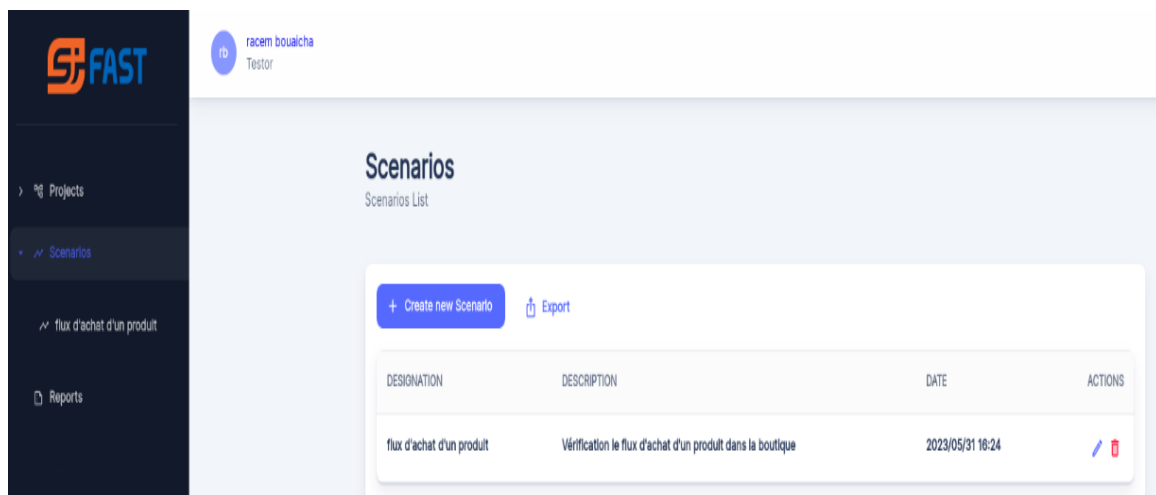


Figure 31: Interface de liste des scénarios

5.2 Interfaces de liste des cas de test exécuté ou à exécuter

Cette interface, schématisée par la figure suivante, affiche la liste des cas de tests exécutés ou à exécuter.

TITLE	PREREQUIREMENTS	EXPECTED RESULT
^ Gérer les images des produits	Les Images doit etre en format PNG	L'affichage doit être en haut à gauche dans la partie front
vérifier le bouton inseret	On doit vérifier que le bouton cli...	Bouton inseret racem bouaicha l'image doit etre importer de PC ✓
vérifier le bouton modifier image	l'image doit etre modifier	modifier image racem bouaicha L'ancien image doit être affiché ✗
Vérifier le bouton supprimer	Vérifier le bouton supprimer d'u...	Supprimer une image Racem bouaicha le bouton doit être cliquable que ... ▶
^ Statistique	Les courbes doivent être en courbe bar et pie	Les statistiques doivent être instantané
Pie	L'affichage de la courbe pie doit é...	la courbe pie racem bouaicha la base de données ne doit pas être ... ✓
bar	L'affichage de la courbe bar doit ...	la courbe bar racem bouaicha la base de données ne doit pas être ... ✓
^ Gérer les catalogues	Il doit exister aux moins deux produits	Les produits doivent être groupé par catégorie dans un tableau
Vérifier le drag and drop des pro...	avec un petit glissement de prod...	drag and drop des catalogues racem bouaicha il doit existe au moins deux produits ✗
créer un catalogue	avec un petit formulaire, on peut...	creation d'un catalogue racem bouaicha pas de pre-condition ✗
^ gérer le panier	Il doit exister au moins un produit	Le panier doit être instantané
Ajouter un produit dans le panier	Vérifier l'ajout d'un produit dans ...	Ajouter un produit dans le panier racem bouaicha pas de précondition ▶

Figure 32: Interface de la liste cas de test exécuté ou à exécuter

5.3 Interfaces de l'exécution de cas de test

Cette interface, schématisée par la figure suivante, affiche l'action d'exécution.

Execute
"Vérifier le bouton supprimé"

support *

☐ Validated
☐ Invalidated

Create

Figure 33: Interface d'exécution de cas de test

5.4 Exemple de rapport

Cette interface, schématisée par la figure suivante, affiche le rapport d'exécution.

le 31/05/2023

Projet de test	Boutique Ecommerce développé avec le CMS Prestashop	Scénario testé	flux d'achat d'un produit
Nombre de sprint	3	Nombre des cas d'utilisation valider	3
Nombre de cas d'utilisation	4	Nombre des cas d'utilisation invalider	3
		Nombre des cas d'utilisation en cours	2

TITLE	PREREQUIREMENTS	EXPECTED RESULT
^ Gérer les images des produits	Les images doit etre en format PNG	L'affichage doit être en haut à gauche dans la partie front
vérifier le bouton inserer	On doit vérifier que le bouton cli...	Bouton inserer
racem bouaicha	l'image doit etre importer de PC	✓
vérifier le bouton modifier image	l'image doit etre modifier	modifier image
racem bouaicha	L'ancien image doit être affiché	✗
Vérifier le bouton supprimer	Vérifier le bouton supprimer d'a...	Supprimer une image
racem bouaicha	le bouton doit être cliquable que ...	▶
^ Statistique	Les courbes doivent être en courbe bar et pie	Les statistiques doivent être instantané
Pie	l'affichage de la courbe pie doit être...	la courbe pie
racem bouaicha	la base de données ne doit pas être...	✓
bar	l'affichage de la courbe bar doit être...	la courbe bar
racem bouaicha	la base de données ne doit pas être...	✓
^ Gérer les catalogues	Il doit exister aux moins deux produits	Les produits doivent être groupé par catégorie dans un tableau
Vérifier le drag and drop des pro...	avec un petit glissement de prod...	drag and drop des catalogues
racem bouaicha	il doit exister au moins deux produits	✗
créer un catalogue	avec un petit formulaire, on peut...	creation d'un catalogue
racem bouaicha	pas de pre-condition	✗
^ gérer le panier	Il doit exister au moins un produit	Le panier doit être instantané
Ajouter un produit dans le panier	Vérifier l'ajout d'un produit dans ...	Ajouter un produit dans le panier
racem bouaicha	pas de précondition	▶

Figure 34: exemple de rapport

6 Conclusion

Ce sprint permet d'exécuter les cas de test en déterminant les scénarios. Chaque scénario va contenir des cas d'utilisation. Chaque cas d'utilisation peut contenir des cas de tests. Ces données seront stockées et exploitées par le testeur.

CHAPITRE 5 CLOTURE DU PROJET

1 Introduction

Ce chapitre marque la clôture de notre projet en mettant en avant les ressources liées aux composants intelligents et en mettant en évidence les choix des techniques, du système de gestion de base de données et des outils utilisés pour la mise en œuvre de notre application. Pour ce faire, nous exposons l'architecture globale de notre système. Ensuite, nous abordons certains outils et logiciels utilisés. Enfin, nous clôturons en procédant à une évaluation du système réalisé.

2 Architecture du système développé

Dans cette section, nous décrivons l'architecture globale utilisée et un scénario de fonctionnement de notre application.

2.1 Architecture globale

L'architecture d'une application joue un rôle crucial dans sa réussite et sa durabilité. Elle permet de structurer les différents éléments du système et de définir leurs interactions. Dans cette section, nous nous exposons l'architecture de notre solution qui s'appuie sur l'architecture à trois niveaux. Dans notre situation, la couche de présentation est réalisée avec React ts, la couche métier est implémentée à l'aide du framework web Symfony, et enfin, la couche de données est représentée par phpMyAdmin.



Figure 35: Architecture global

L'architecture de l'application repose sur Symfony pour la couche de traitement, où le router agit comme point d'entrée pour les requêtes HTTP et les dirige vers les contrôleurs. Ces contrôleurs encapsulent le logique métier de chaque route et renvoient une réponse. Les services sont responsables de la logique métier de l'application, tandis que les modèles représentent la couche d'abstraction de la base de données PhpMyAdmin. En ce qui concerne la couche de présentation, l'application utilise React, avec des composants qui construisent l'interface utilisateur, ainsi qu'un système de routage qui interagit avec le modèle en appelant les méthodes exposées. Enfin, la couche d'accès aux données est gérée par PhpMyAdmin, qui offre des fonctionnalités avancées pour la gestion de systèmes de données complexes, permettant de gérer efficacement un grand nombre d'utilisateurs grâce à un SGBD performant et puissant. Cette architecture favorise la maintenabilité et la testabilité de l'application, en respectant les bonnes pratiques de développement.

2.2 Fonctionnement d'application

Nous illustrons, par la figure 36, la démarche suivie pour exécuter notre solution.

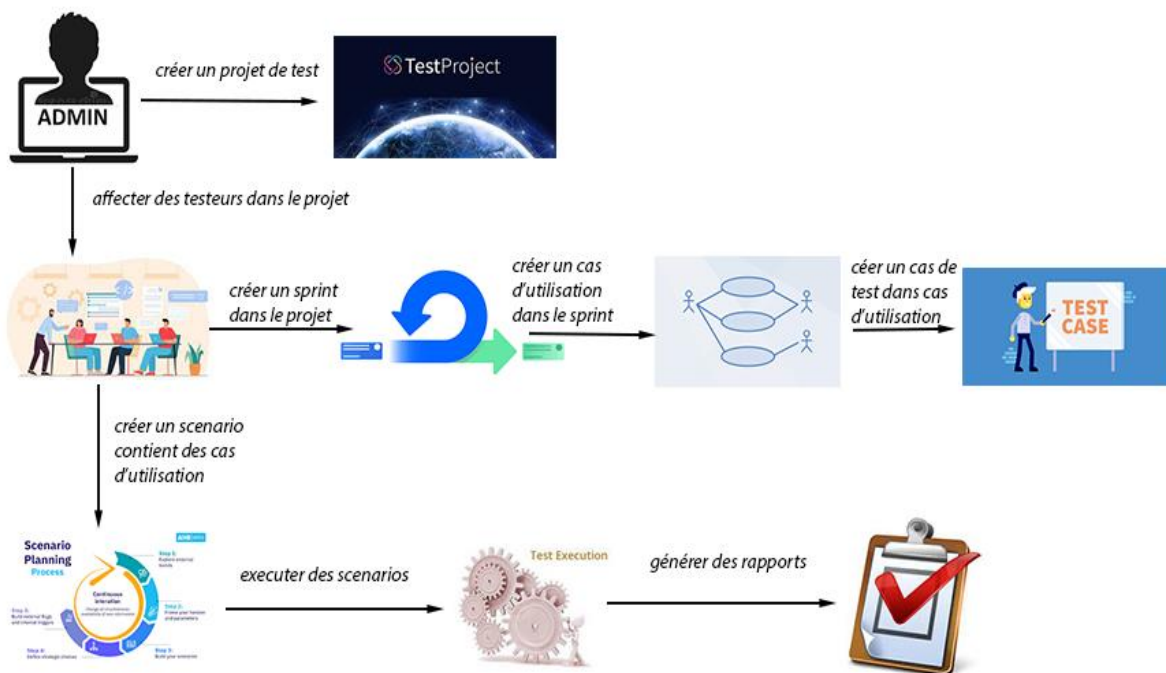


Figure 36: Démarche générale

Le fonctionnement de notre système débute par l'affectation d'un projet à tester par l'administrateur à un testeur. Dans notre cas, nous allons s'intéresser aux tests fonctionnels proposés par notre encadreur. Ensuite, le testeur prépare le plan du test. Il regroupe les cas d'utilisation (modules) dans un sprint. Puis, il regroupe les cas de test dans un cas d'utilisation.

Enfin, il exécute ce plan de test selon un scénario. Comme résultat, un rapport est généré contenant les états des différents cas de test (validé, non validé, en cours). Le testeur peut lancer par la suite une exécution les cas de test qu'ils sont déjà inclus dans les cas d'utilisation et finalement le testeur peut générer des rapports qui contiennent toutes les informations de projet de test.

3 Outils et environnements de travail

La phase de réalisation de notre projet s'est basée sur une diversité d'outils logiciels et en utilisant une variété de langages de développement.

3.1 Environnement logiciel

- ✓ **Visual Studio Code** : un éditeur de code source qui peut être utilisé avec une variété de langages de programmation [7].
- ✓ **Git** : est un logiciel libre de gestion de versions décentralisée. Nous l'avons employé pour assurer le suivi des versions de notre code tout au long du développement et pour conserver l'historique de toutes les modifications effectuées [8].
- ✓ **Postman** : est un client REST qui facilite et accélère les tests des requêtes HTTP. Il propose une interface graphique permettant de visualiser plus clairement les réponses des requêtes envoyées vers le backend, ce qui rend le processus de test rapide et simple [9].

3.2 Environnement de développement

- ✓ **Docker** : permet d'embarquer une application dans un ou plusieurs containers logiciels qui pourra s'exécuter sur n'importe quel serveur machine, qu'il soit physique ou virtuel [10].
- ✓ **Type Script** : est un langage de programmation créé par Microsoft en 2012. Son objectif principal est d'améliorer la productivité lors du développement d'applications complexes. Cependant, ce langage introduit des fonctionnalités optionnelles telles que le typage et la programmation orientée objet. Il n'est pas nécessaire d'inclure des bibliothèques supplémentaires pour bénéficier de ces fonctionnalités [11].

- ✓ **PHP 8** : le PHP, pour Hypertext Preprocessor, désigne un langage informatique, ou un langage de script, utilisé principalement pour la conception de sites web dynamiques. Il s'agit d'un langage de programmation sous licence libre qui peut donc être utilisé par n'importe qui de façon totalement gratuite [12].
- ✓ **Symfony** : est un framework qui représente un ensemble de composants (aussi appelés librairies) PHP autonomes qui peuvent être utilisés dans des projets web privé ou open source [13].
- ✓ **React** : est une bibliothèque JavaScript frontale à code source ouvert permettant de créer des interfaces utilisateur ou des composants d'interface utilisateur. Elle est maintenue par Facebook et une communauté de développeurs individuels et d'entreprises [14].
- ✓ **PhpMyAdmin** : est un logiciel libre écrit en PHP qui a pour mission de s'occuper de l'administration d'un serveur de base de données MySQL ou MariaDB. Vous pouvez utiliser phpMyAdmin pour réaliser la plupart des tâches d'administration, ceci incluant la création de base de données, l'exécution de demandes, et l'ajout de comptes utilisateur [15].
- ✓ **JSON** : JavaScript Object Notation (JSON) est un format de données textuel dérivé de la notation des objets du langage JavaScript. Il concurrence XML pour la représentation et la transmission d'information structurée [16].

4 Evaluation

Dans la littérature, il existe quelques applications qui permettent d'éviter la file d'attente, comme par exemple « **Test Collab** ». Cependant, cette application la plupart de ses services est payante comme l'exportation des rapports, et des autres services n'est pas accessible comme la maintenance de projet de test et la création des sprints de test. Et pour cela nous proposons notre solution qui permet au testeur de faire le test fonctionnel, regrouper les modules de test avec les sprints et exporter des rapports de test plus simple et facilement d'exploiter.

5 Conclusion

Dans ce chapitre, nous avons tout d'abord décrit l'architecture technique et matérielle de notre application dans le cadre de notre projet. Ensuite, la deuxième partie a été consacrée à la présentation des logiciels utilisés pour le développement de notre système.

CONCLUSION GENERALE

Les sociétés de développement utilisent actuellement des méthodes AGILE pour réaliser des logiciels pour exécuter des services proposés par les clients. Ces méthodes ont donné satisfaction sur le marché. Elles se basent sur les compétences humaines et favorisent les relations et la communication entre les personnes. SCRUM est la plus connue. Elle se base sur le découpage d'un projet en fonctionnalités par le product owner. Les fonctionnalités sont regroupées en sprints par le scrum master et l'équipe SCRUM. Chaque sprint est développé en équipe selon le cycle de développement. L'ensemble des tâches effectuées par les membres de l'équipe SCRUM doivent être testées et validées. Les tests logiciels sont très importants pour assurer la qualité des produits et améliorer leur rentabilité. Cette étape est dans la plupart des cas n'est pas considérée ce qui provoque différents types des erreurs.

C'est pourquoi, nous avons développé une application pour gérer les tests proposés par le responsable (les tests fonctionnels). Notre application est générique. Elle prend comme paramètre les projets de notre société Sifast. Ces projets seront affectés aux testeurs. En utilisant SCRUM, ces projets sont découpés en sprints. Les sprints sont découpés en cas d'utilisation. Les cas d'utilisation sont découpés en cas de tests. Le testeur va proposer un scénario pour exécuter des tests et générer un rapport. L'application proposée permet d'assister les testeurs et vérifier les données de tests.

Nous avons essayé de suivre les recommandations proposées par notre société encadrante pour exécuter les tâches proposées. Néanmoins, ce projet nécessite plus de temps pour mettre en place tous les tests logiciels. C'est pourquoi, nous proposons comme perspectives d'intégrer notre application dans la plateforme de la société (en cours) pour exploiter les données des projets de différents systèmes existants et ajouter d'autres fonctionnalités pour supporter les autres types de tests logiciels. Enfin, nous proposons d'améliorer le rapport généré en ajoutant des statistiques et des courbes analysant les résultats des tests obtenus.

REFERENCES BIBLIOGRAPHIQUES

- [1] <https://www.definitions-marketing.com/definition/assurance-qualite/>
- [2] <https://www.eurotechconseil.com/blog/importance-assurance-qualite-dans-developement-logiciel>.
- [3] <https://fr.theastrologypage.com/software-testing>
- [4] <https://www.journaldunet.fr/web-tech/guide-de-l-entreprise-digitale/1443834-scrum-maitriser-le-framework-star-des-methodes-agiles/>
- [5] <https://bubbleplan.net/blog/wp-content/uploads/2018/05/430.jpeg>
- [6] « UML 2 pour les bases des données », Christian Soutou, édition Eyrolles, 2007 »
- [7] <https://visualstudio.microsoft.com/fr/>
- [8] <https://www.atlassian.com/fr/git/tutorials/what-is-git>
- [9] <https://blog.webnet.fr/presentation-de-postman-outil-multifonction-pour-api-web/>
- [10] <https://www.journaldunet.fr/web-tech/guide-de-l-entreprise-digitale/1146290-docker-definition-docker-compose-docker-hub-docker-swarm-160919/>
- [11] <https://blog.cellenza.com/developpement-specifique/introduction-a-typescript/>
- [12] <https://www.journaldunet.fr/web-tech/dictionnaire-du-webmastering/1203597-php-hypertext-preprocessor-definition/>
- [13] <https://www.pure-illusion.com/lexique/definition-de-symfony>
- [14] <https://www.50a.fr/0/react>
- [15] <https://docs.phpmyadmin.net/fr/latest/intro.html>
- [16] <https://www.json.org/json-en.html>

الخلاصة: في سياق مشروع نهاية الدراسة، قمنا بتطوير تطبيق لإدارة حالات الاختبار داخل شركة « SiFAST ». هذا التطبيق يساعد المختبرين لتنفيذ خطط الاختبار على المشاريع المتأثرة بالمسؤول، وفقاً لطريقة SCRUM ركزنا على الاختبار الميزات التي تقدمها الشركة. تطبيقنا قابل للتوسيع ويمكن أن يكون كذلك مدمج في النظام الأساسي الحالي.

المفاتيح: اختبار البرمجيات، وحالات الاستخدام، وحالات الاختبار.

Développement d'une application de gestion de cas de test et de génération des rapports de tests

Résumé : Dans le contexte de projet de fin d'étude, nous avons développé une application de gestion de cas tests au sein de la société SiFAST. Cette application permet d'aider les testeurs d'exécuter des plans des tests sur les projets affectés par l'administrateur, selon la méthode SCRUM. Nous avons concentré sur les tests fonctionnels proposés par la société. Notre application est extensible et peut être intégrée dans la plateforme existante.

Mots clés : Tests logiciels, sprint, cas d'utilisation, cas de test.

Development of an application for managing test cases and generating test reports

Abstract: In the context of an end-of-study project, we have developed an application of management of test cases within the SiFAST Company. This application helps testers to execute test plans on projects affected by the administrator, according to the SCRUM method. We focused on testing features offered by the company. Our application is extensible and can be integrated into the existing platform.

Keywords: Software testing, sprint, use case, test case.